

ALMA MATER EUROPAEA

VS študijski program: Spletne in informacijske tehnologije

Predmet: Računalništvo v oblaku in mobilne rešitve

PROJEKTNA NALOGA

Implementacija in avtomatizacija brezstrežnega (Serverless) bota v okolju AWS

Študent: Enej Volmut

Letnik: 3. letnik

Rače, 02. 05. 2026

Leto vpisa: 2025/26

KAZALO

1. UVOD IN CILJI PROJEKTA	3
2. OPIS UPORABLJENIH TEHNOLOGIJ	4
3. IZVEDBA PROJEKTA	5
4. ZAKLJUČEK	29
5. VIRI	30

1. UVOD IN CILJI PROJEKTA

Za svojo projektno nalogo sem si izbral področje »serverless« arhitekture in avtomatizacije procesov (CI/CD). Glavni cilj je bil izdelati Discord bota, ki za svoje delovanje ne potrebuje klasičnega virtualnega strežnika (kot je npr. EC2), ki bi moral teči 24/7. Namesto tega sem uporabil storitev AWS Lambda, kjer se koda sproži le takrat, ko je dejansko potrebna.

Glavni cilji, katere sem si zadal, so bili:

- Vzpostavitev delujočega Discord bota v Python jeziku.
- Uporaba AWS Lambda za gostovanje kode.
- Avtomatizacija posodabljanja bota preko GitHub Actions (vsak »push« na GitHub avtomatsko posodobi bota na AWS-ju).

2. OPIS UPORABLJENIH TEHNOLOGIJ

Pri izdelavi projekta sem izbral tehnologije, ki omogočajo visoko stopnjo avtomatizacije in kjer so nižji stroški vzdrževanja. Spodaj so podrobneje opisana glavna orodja in jeziki:

Python (3.12): To je bil moj glavni programski jezik, v katerem sem napisal vso logiko bota. Za komunikacijo z Discordovim API-jem sem uporabil knjižnjico »discord.py«.

PyCharm: To je bilo moje glavno razvojno okolje (IDE) za pisanje Python kode. Ta program sem izbral, ker mi je omogočil lažje pisanje in testiranje samega bota, preden sem ga poslal v oblak.

AWS Lambda: Storitev, ki omogoča »brezstrežno« delovanje. Za Lambdo sem se odločil, ker ni potrebno nastavljanje celotnega strežnika in zaračunava le čas izvajanja kode, kar je stroškovno najbolj učinkovito.

AWS IAM: To storitev sem uporabil za pridobitev varnostnih ključev (Access Key ID in Secret Access Key) iz kreiranega uporabnika. Ta ključa sta nujna za varno avtomatizirano povezavo med GitHubom in AWS-jem.

Discord Developer Portal: Spletna nadzorna plošča, kjer sem registriral bota, pridobil njegov unikatni žeton (Token) in nastavil dovoljenja, da je bot lahko bral in pošiljal sporočila na strežniku.

Visual Studio Code: Okolje, kjer sem konfiguriral .yaml datoteko za avtomatizacijo.

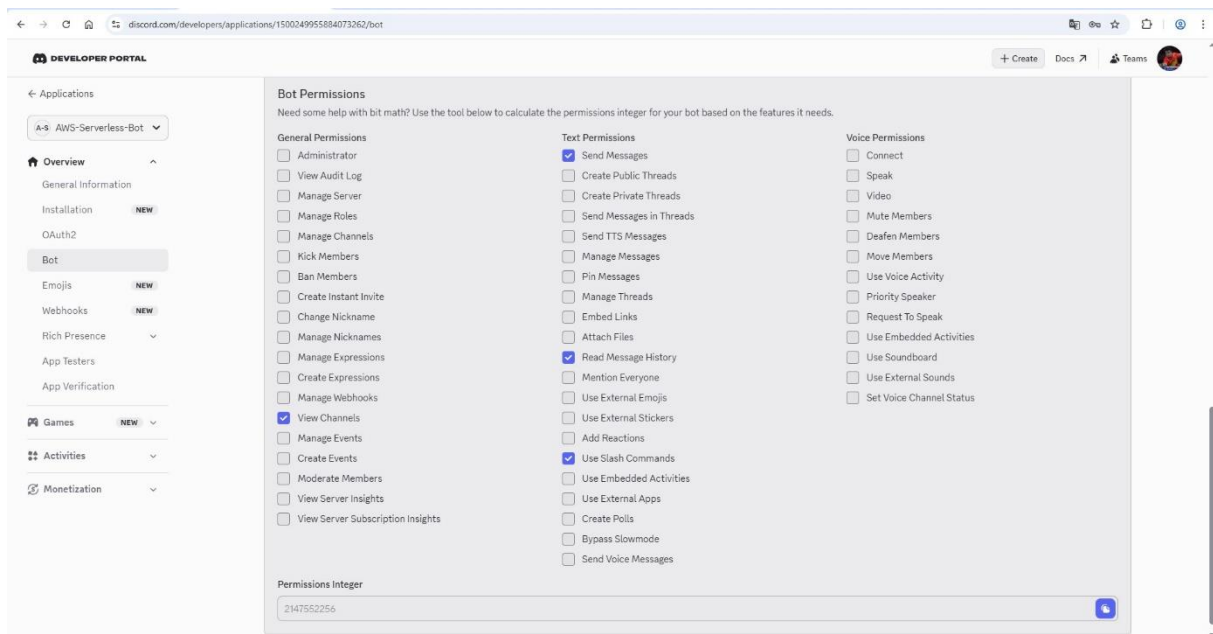
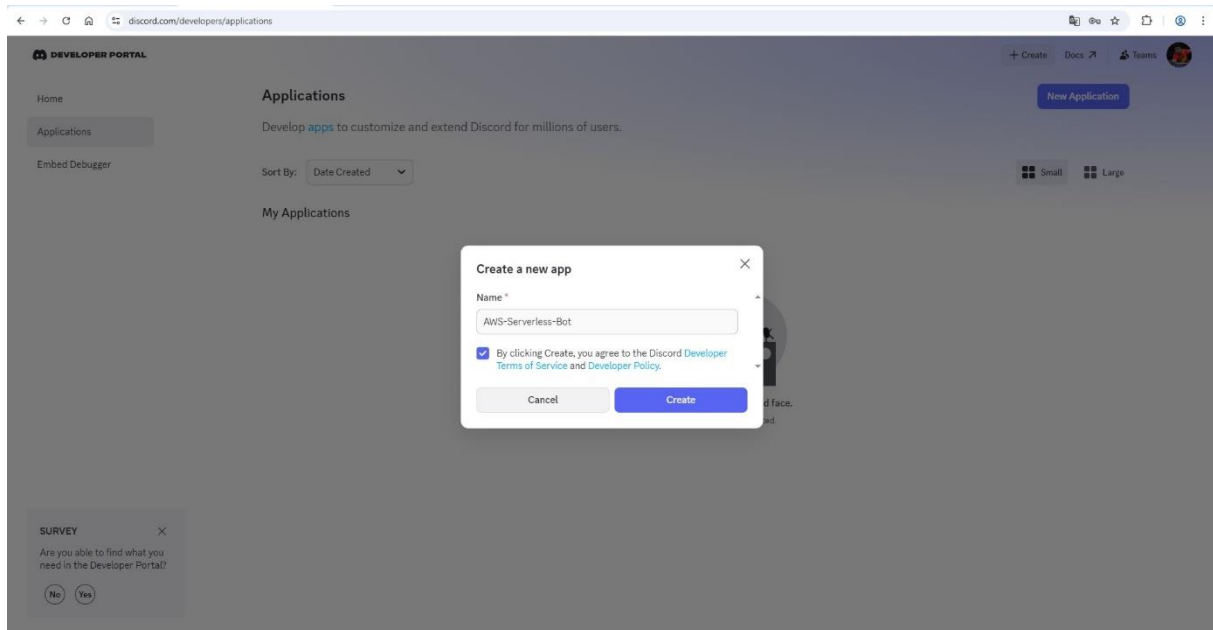
GitHub & GitBash: GitHub sem uporabil za shranjevanje kode v oblaku. GitBash pa za pošiljanje (z ukazi »git commit« in »git push«) lokalnih datotek na GitHub.

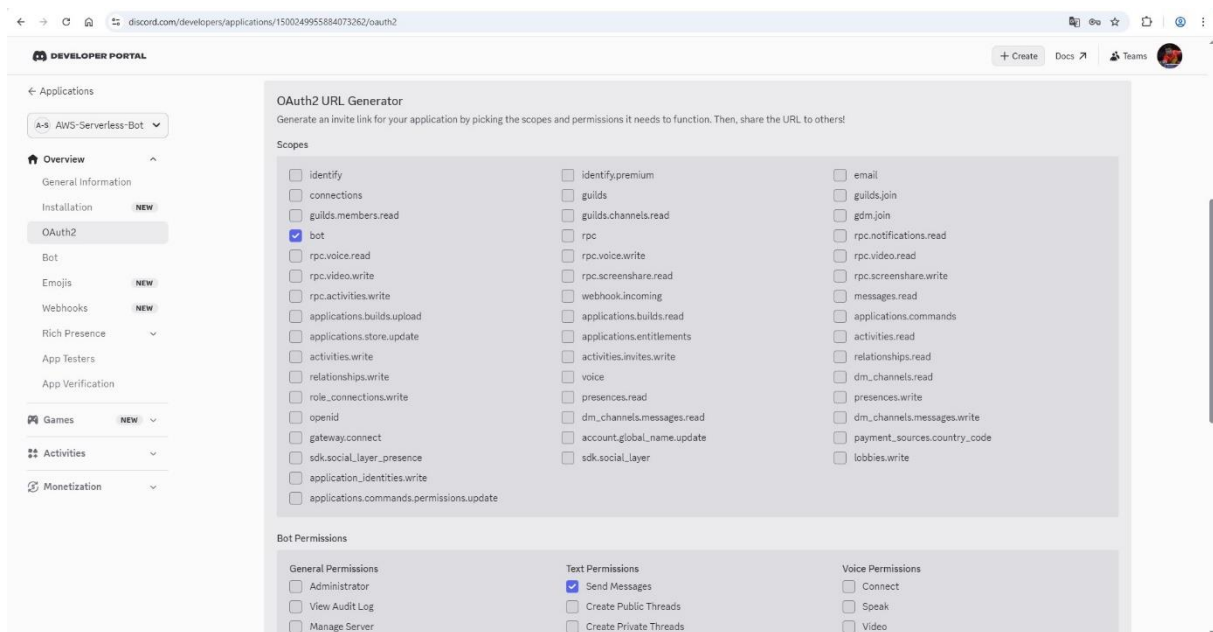
Windows CMD: Na začetku sem ga uporabil za ustvarjanje map in nalaganje knjižnice »discord.py« na lokalni računalnik.

GitHub Actions: Orodje za avtomatizacijo. Z uporabo .yaml datoteke sem vzpostavil pravila, ki se ob vsakem ukazu »git push« koda samodejno zapakira v .zip datoteko, namesti knjižnice in vse skupaj naloži na AWS Lambdo.

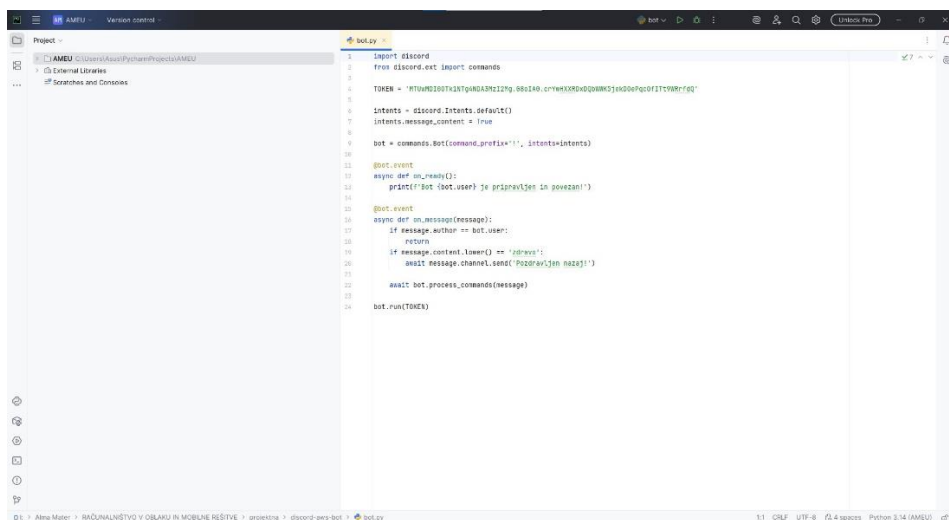
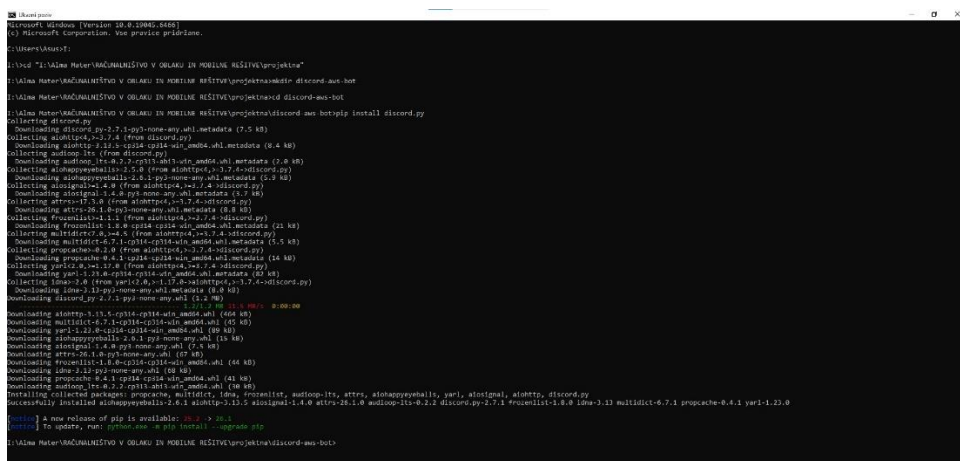
3. IZVEDBA PROJEKTA

Proces sem začel na portalu za razvijalce Discord, kjer sem registriral novo aplikacijo ter pridobil ključne poverilnice za identifikacijo. V zavihku za nastavitve bota sem general avtentikacijski žeton in vklopil vse potrebne privilegirane namene, ki botu sploh omogočajo interakcijo s strežniki in uporabniki.

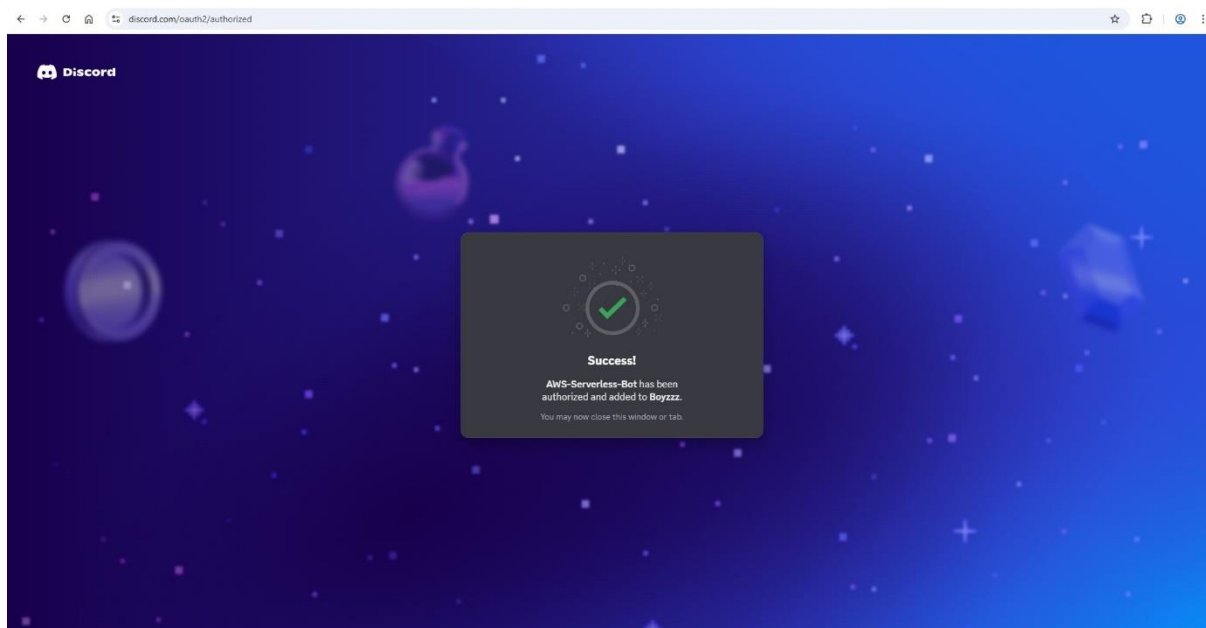




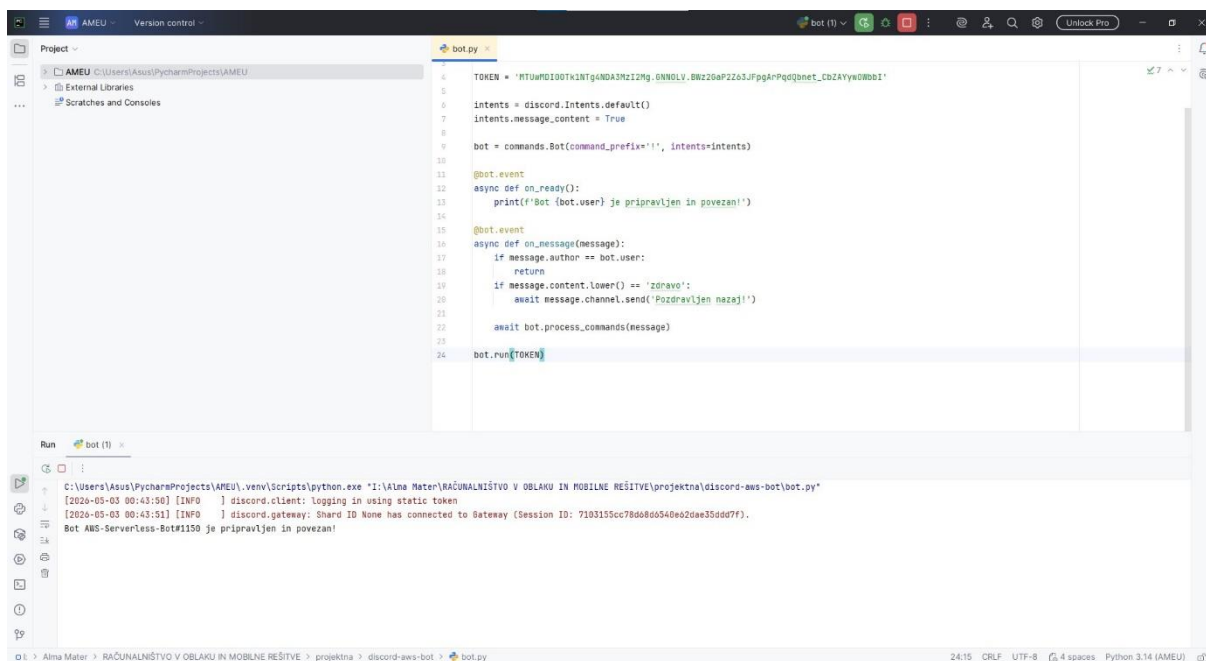
Ko sem pripravil digitalno identiteto bota, sem v CMD Windows naložil knjižnjico »discord.py« in v urejevalniku PyCharm ustvaril glavno programsko datoteko z imenom bot.py.



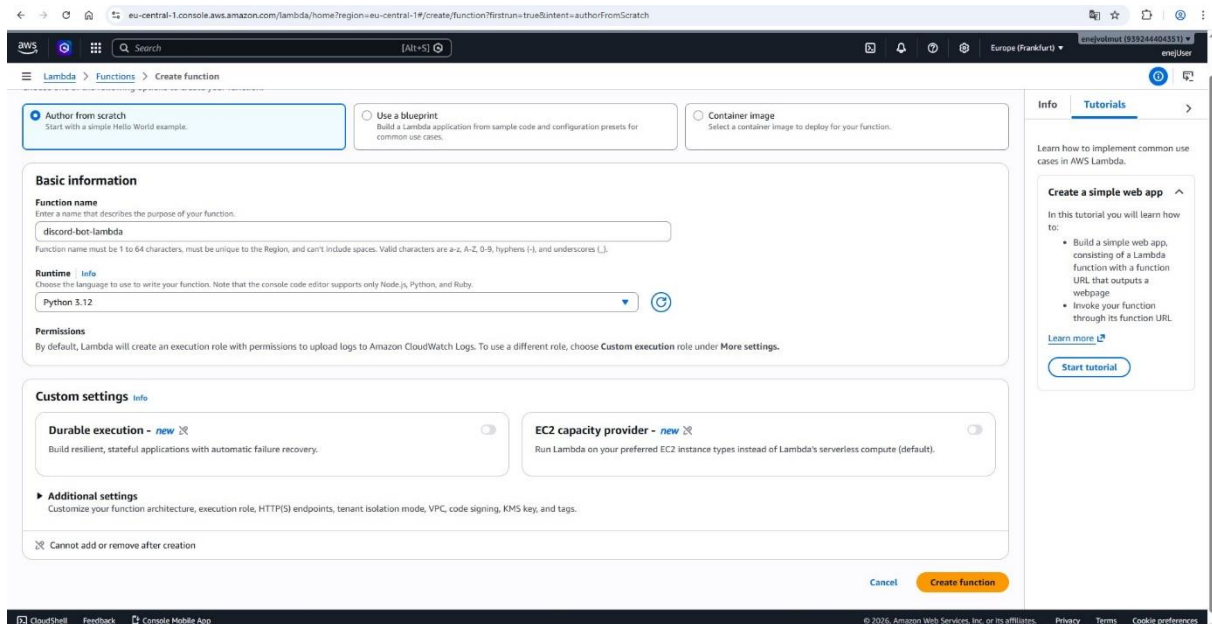
Po generiranju povezave za avtorizacijo sem bota uspešno dodal na svoj izbrani Discord strežnik, kar je potrdilo sistemsko sporočilo o uspešni povezavi aplikacije z imenom AWS-Serverless-Bot. Kasneje sem strežnik preimenoval v »S1«.



Po uspešnem lokalnem testiranju sem začel z vzpostavitvijo infrastrukture na platformi AWS Lambda, ki mi je omogočila poganjanje kode brez vzdrževanja lastnega strežnika.

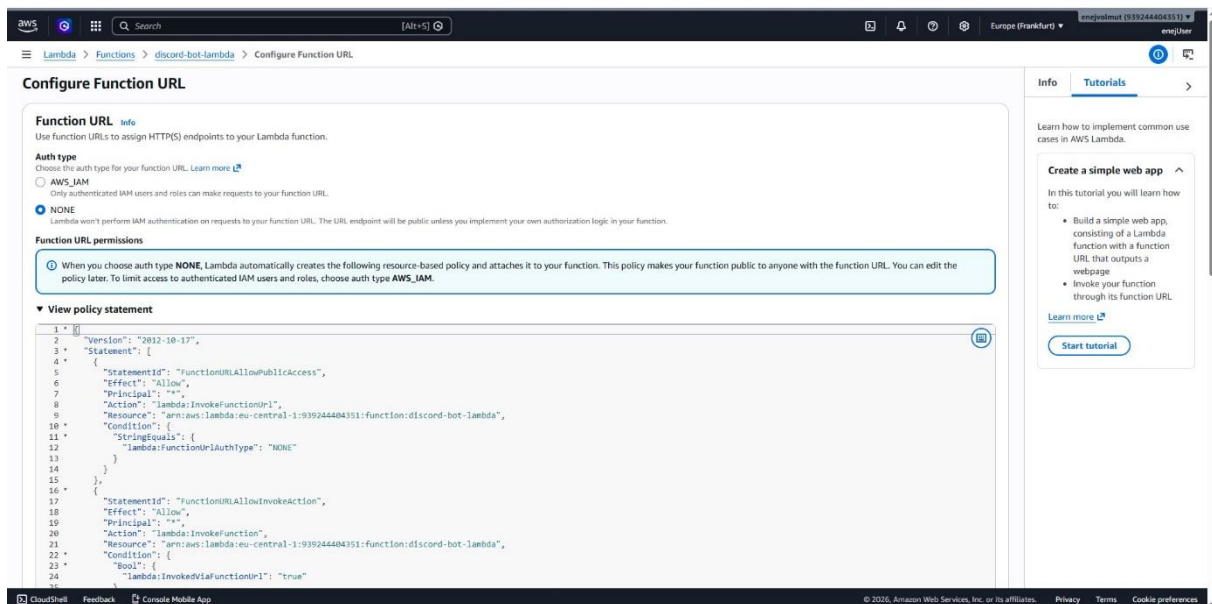


V nadzorni plošči AWS sem kreiral novo funkcijo z uporabo okolja Python 3.12 in določil osnovne izvajalne parametre.



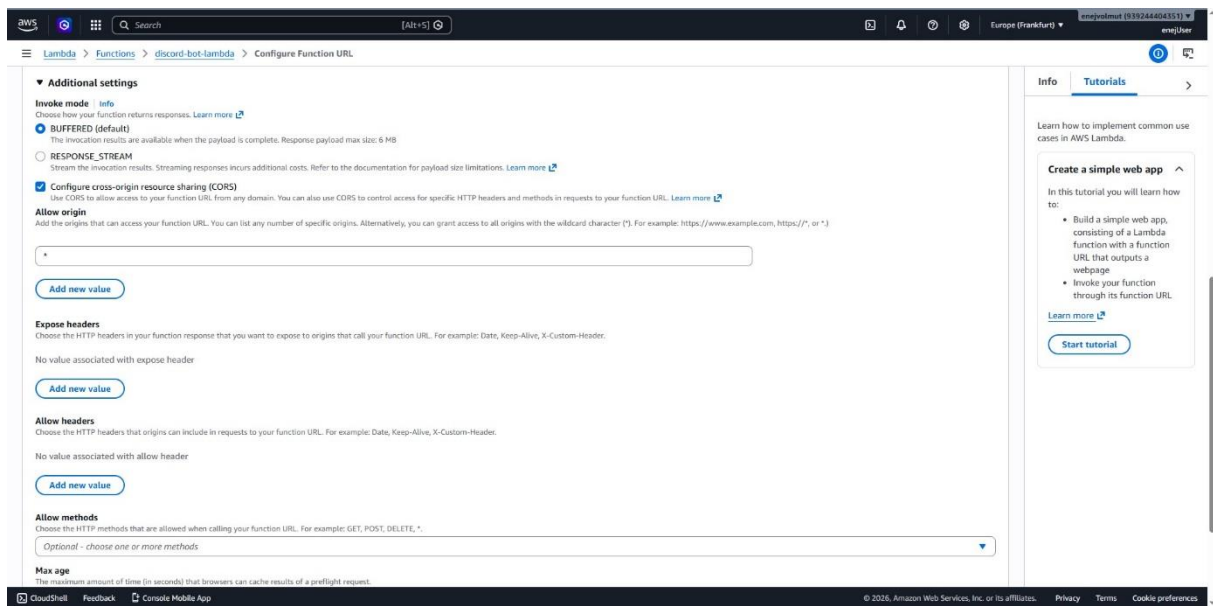
The screenshot shows the AWS Lambda 'Create function' page. The 'Author from scratch' tab is selected. The 'Function name' is 'discord-bot-lambda'. The 'Runtime' is 'Python 3.12'. The 'Permissions' section shows the default execution role. The 'Custom settings' section has 'Durable execution' and 'EC2 capacity provider' toggled off. The 'Create function' button is visible at the bottom right.

Da bi bot lahko prejel ukaze neposredno z Discordovih strežnikov, sem funkciji dodelil javen URL naslov.

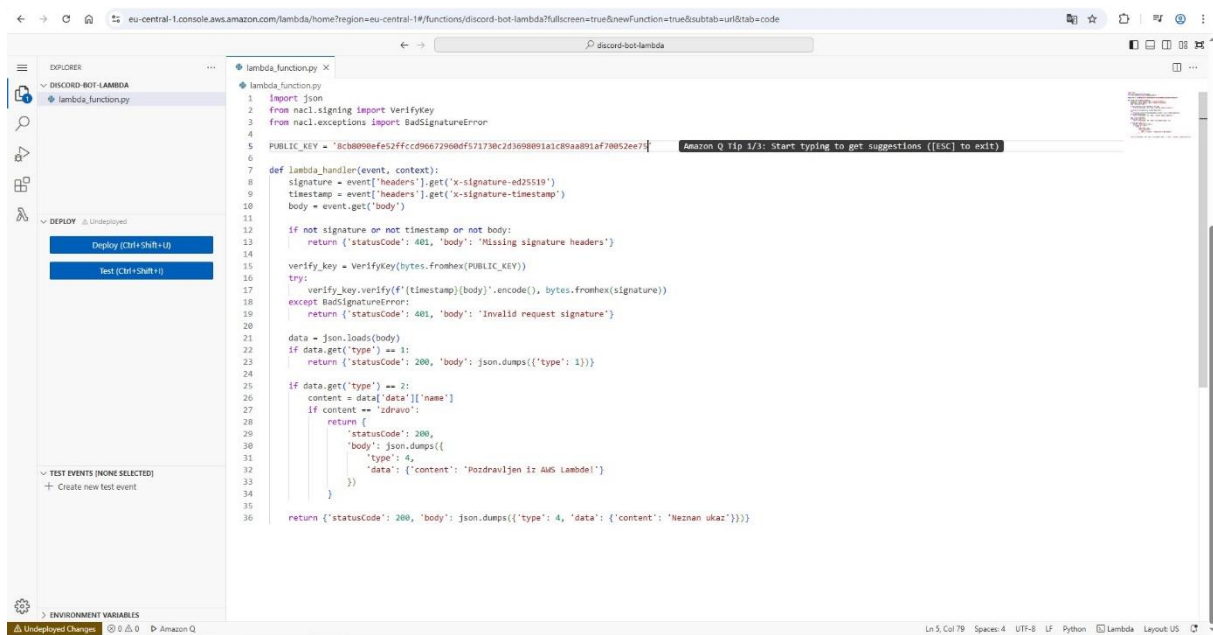


The screenshot shows the AWS Lambda 'Configure Function URL' page. The 'Auth type' is set to 'NONE'. The 'Function URL permissions' section shows a policy statement that allows public access to the function URL. The 'View policy statement' section shows the JSON policy document.

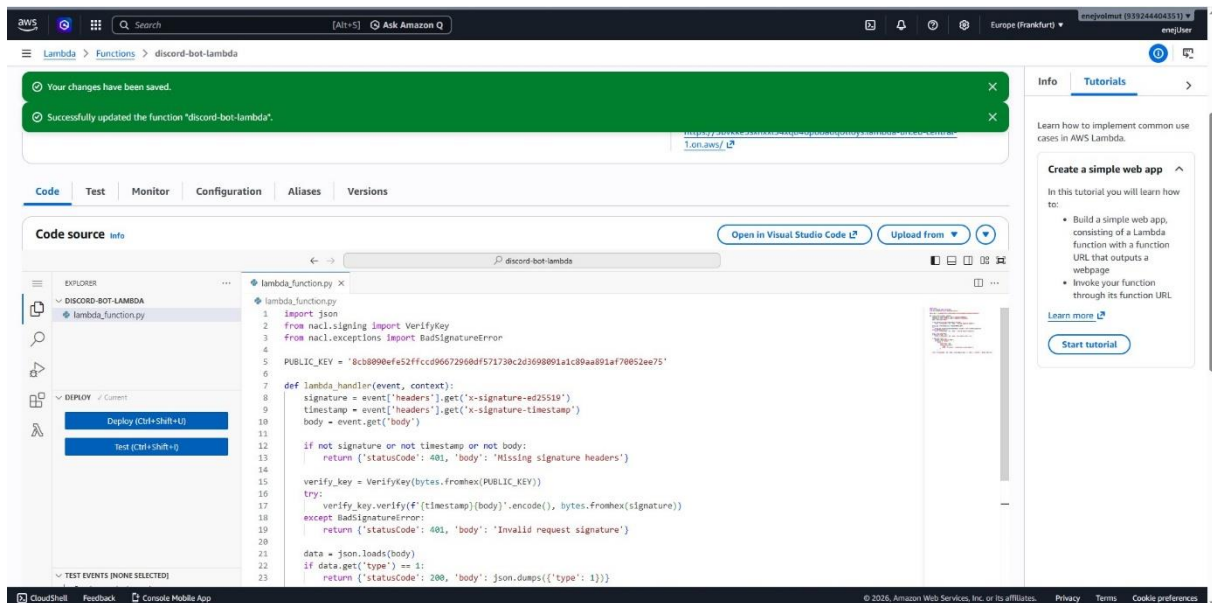
```
1 *
2 {
3   "Version": "2012-10-17",
4   "Statement": [
5     {
6       "Sid": "FunctionURLAllowPublicAccess",
7       "Effect": "Allow",
8       "Principal": "*",
9       "Action": "lambda:InvokeFunctionUrl",
10      "Resource": "arn:aws:lambda:eu-central-1:939244404351:function:discord-bot-lambda",
11      "Condition": {
12        "StringEquals": {
13          "lambda:FunctionUrlAuthType": "NONE"
14        }
15      }
16    },
17    {
18      "Sid": "FunctionURLAllowInvokeAction",
19      "Effect": "Allow",
20      "Principal": "*",
21      "Action": "lambda:InvokeFunction",
22      "Resource": "arn:aws:lambda:eu-central-1:939244404351:function:discord-bot-lambda",
23      "Condition": {
24        "Bool": {
25          "lambda:InvokeFunctionUrl": "true"
26        }
27      }
28    }
29  ]
30 }
```

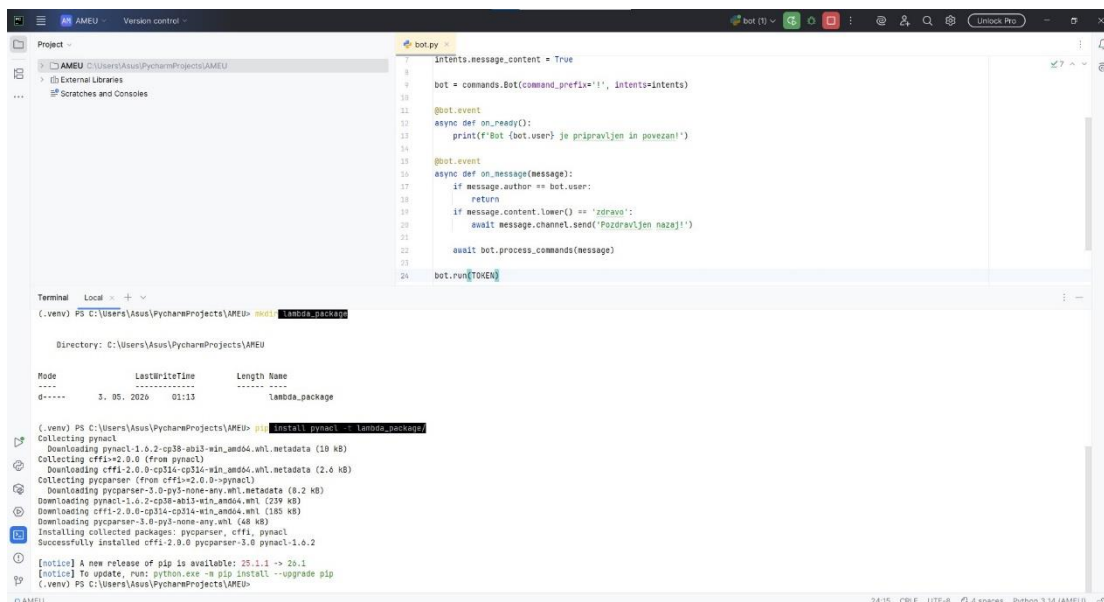
V urejevalniku kode sem pripravil vsebino datoteke `lambda_function.py`, kjer sem uvozil potrebne knjižnice za verifikacijo Discordovih zahtevkov in definiral glavno funkcijo `lambda_handler`.



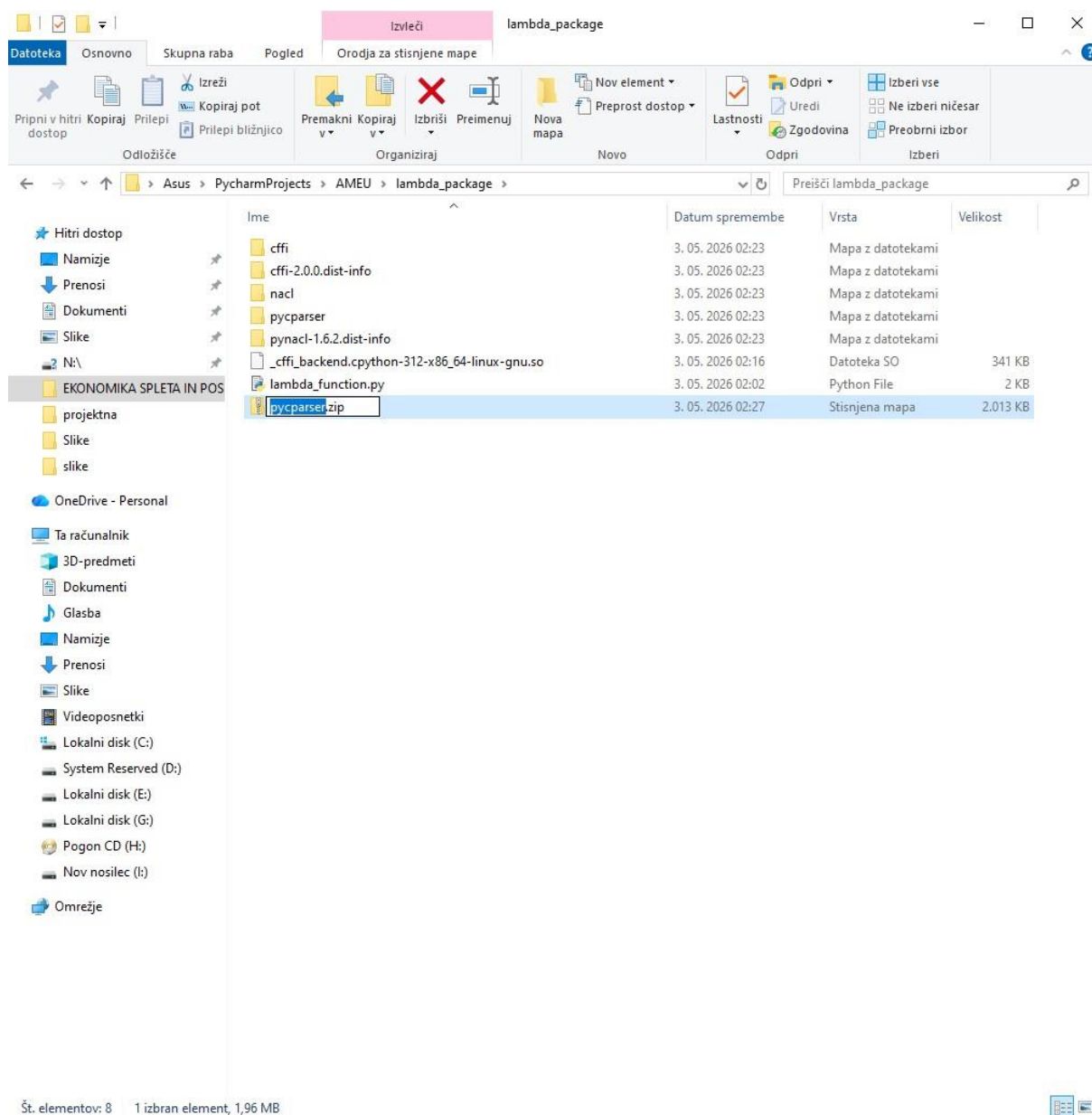
Po vsaki spremembi v spletnem urejevalniku AWS sem kodo shranil in jo s klikom na gumb Deploy objavil, s čimer sem zagotovil, da so zadnje popravke sprejele vse instance funkcije.



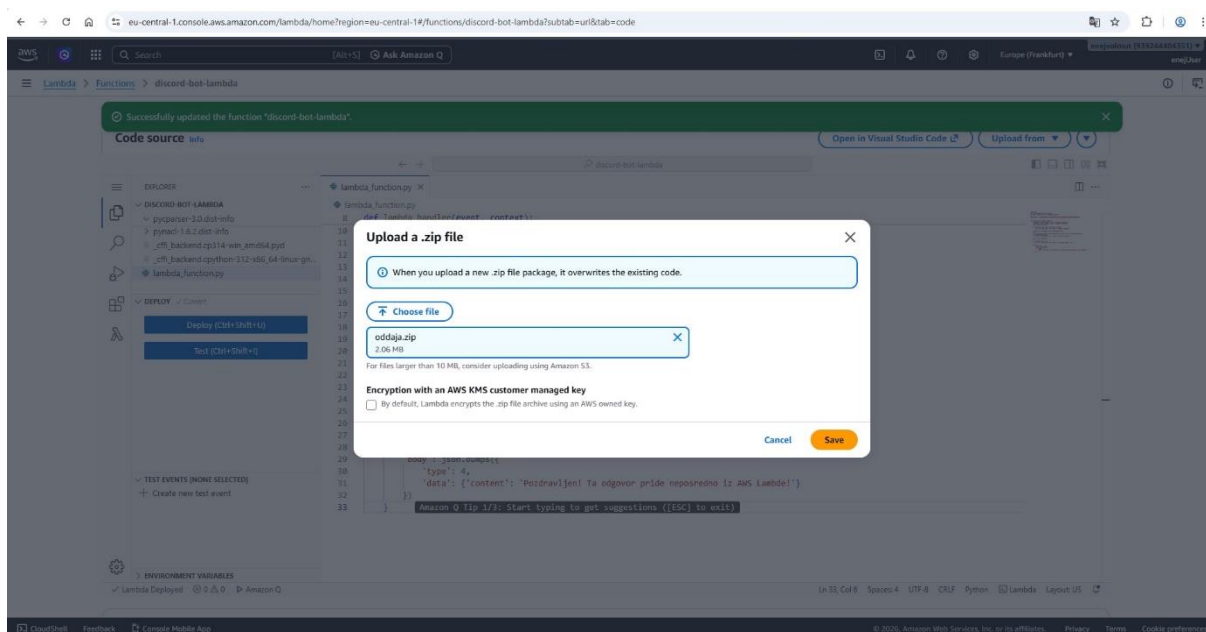
V lokalnem terminalu sem najprej ustvaril novo mapo z ukazom »mkdir lambda_package«, nato pa vanjo s pomočjo orodja pip namestil knjižnico pynacl, ki je nujna za varno komunikacijo z Discordom.



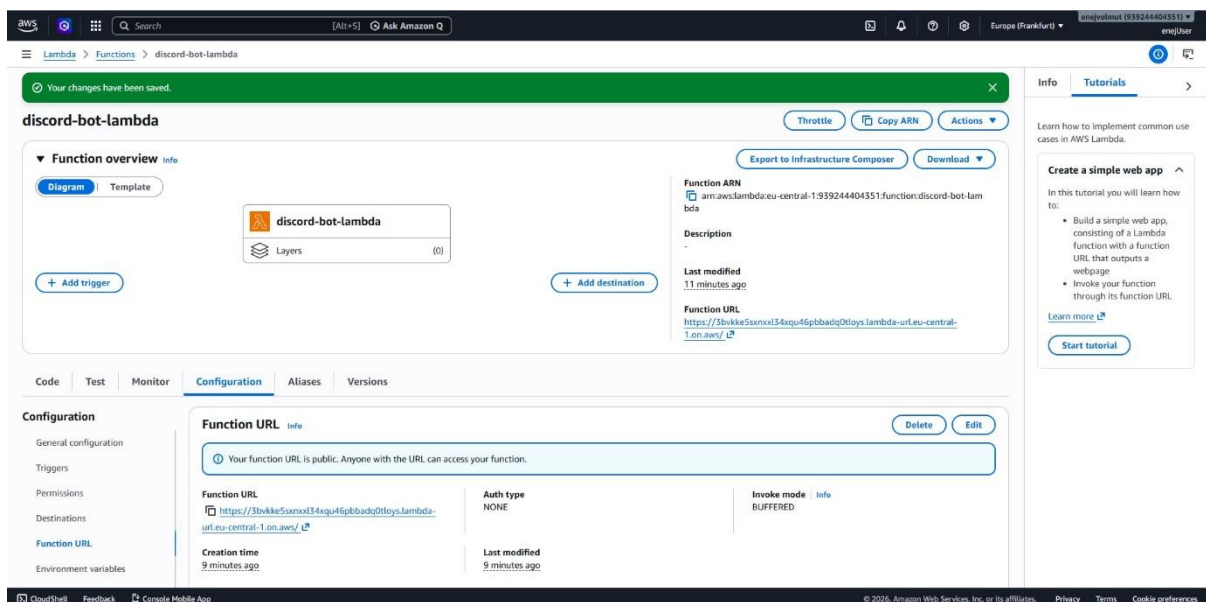
V raziskovalcu datotek sem preimenoval mapo na »oddaja.zip« kjer so se poleg moje programske kode nahajale tudi vse nameščene datoteke.



Nato sem preko nadzorne plošče AWS Lambda ročno naložil v oblak z izbiro možnosti »Upload from .zip file« in naložil stisnjeno datoteko.



Ta generirani naslov sem nato skopiral in ga vnesel v nastavitve bota na strani Discord pod razdelek za interakcije, s čimer sem vzpostavil aktivno komunikacijsko pot med obema sistemoma.



←

→

🔍

discord.com/developers/applications/1500249955884073262/information

🔔

👤

☆

📁

⚙️

DEVELOPER PORTAL

+ Create

Docs ↗️

Teams 👤

👤

← Applications

A-S AWS-Serverless-Bot ▾

Overview

General Information

Installation NEW

OAuth2

Bot

Emojis NEW

Webhooks NEW

Rich Presence ▾

App Testers

App Verification

Games NEW ▾

Activities ▾

Monetization ▾

Public Key

8c80990efe62ffcc096672960df671730c2d3698091a1c09aa891af70052ee75

Copy

Install Count

This is an approximated number of servers and users that have installed your application. These numbers are updated daily.

👤 0 Servers

👤 0 Individual Users

Authorization Count

This is an approximated number of users that have OAuth2 authorizations for your application. This number is updated daily.

👤 0 Individual Users

Interactions Endpoint URL

You can optionally configure an interactions endpoint to receive interactions via HTTP POSTs rather than over Gateway with a bot user.

https://3bvkke5sxxxt34xqu46pbbaq0tloys.lambda-url.eu-central-1.on.aws/

Linked Roles Verification URL

You can configure a verification URL to enable your application as a requirement in a server role's Link settings.

https://nice-example.local/verify-user

Terms of Service URL

A link to your application's Terms of Service

https://my-cool-app.com/terms-of-service

Privacy Policy URL

A link to your application's Privacy Policy

https://my-cool-app.com/privacy-policy

Delete App

←

→

🔍

discord.com/developers/applications/1500249955884073262/information

🔔

👤

☆

📁

⚙️

DEVELOPER PORTAL

+ Create

Docs ↗️

Teams 👤

👤

← Applications

A-S AWS-Serverless-Bot ▾

Overview

General Information

Installation NEW

OAuth2

Bot

Emojis NEW

Webhooks NEW

Rich Presence ▾

App Testers

App Verification

Games NEW ▾

Activities ▾

Monetization ▾

All your edits have been carefully recorded.

App Icon

A-S

Dimensions: 1024x1024

Aspect Ratio: 1:1

File Types: PNG, GIF, JPG, WEBP

Max Size: 10MB

Name

AWS-Serverless-Bot

Description (maximum 400 characters)

Your description will appear in the About Me section of your bot's profile.

Tags (maximum 5)

Add up to 5 tags to describe the content and functionality of your application.

Application ID

1500249955884073262

Copy

Public Key

8c80990efe62ffcc096672960df671730c2d3698091a1c09aa891af70052ee75

Copy

Install Count

This is an approximated number of servers and users that have installed your application. These numbers are updated daily.

👤 0 Servers

👤 0 Individual Users

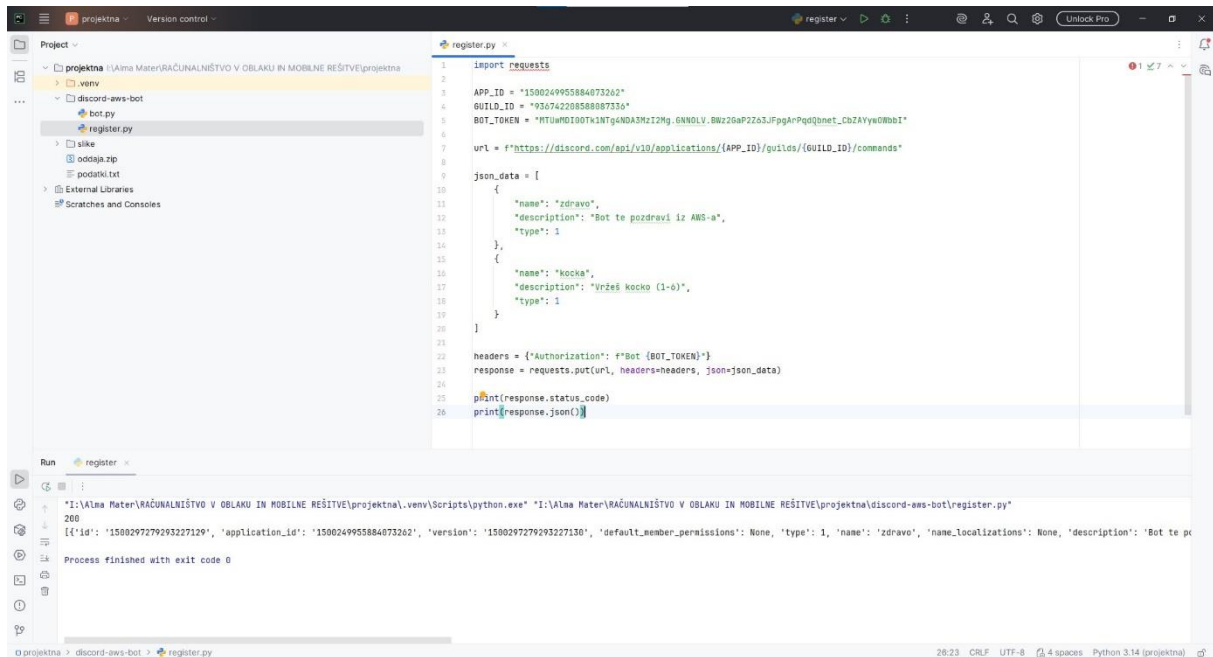
Authorization Count

This is an approximated number of users that have OAuth2 authorizations for your application. This number is updated daily.

...

13

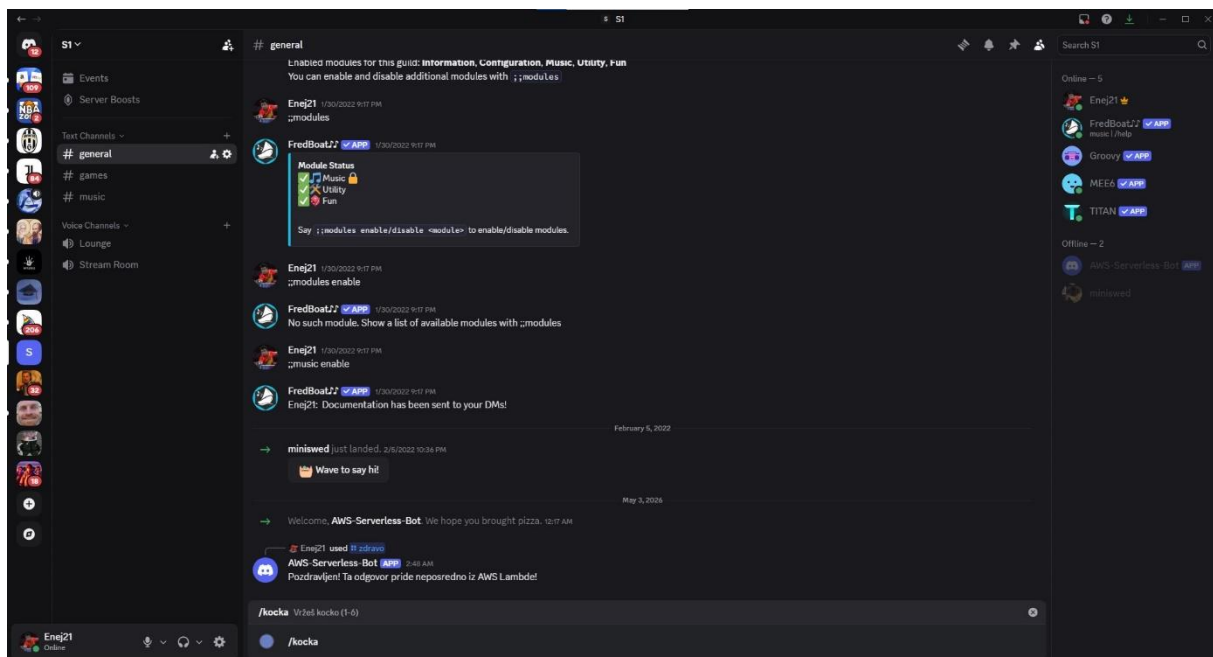
V PyCharmu sem pripravil in zagnal skripto register.py, ki je z uspešnim odgovorom 201 potrdila registracijo ukazov na Discordovih strežnikih, kar mi je nato omogočilo uspešno testiranje delovanja neposredno v Discord klepetu.

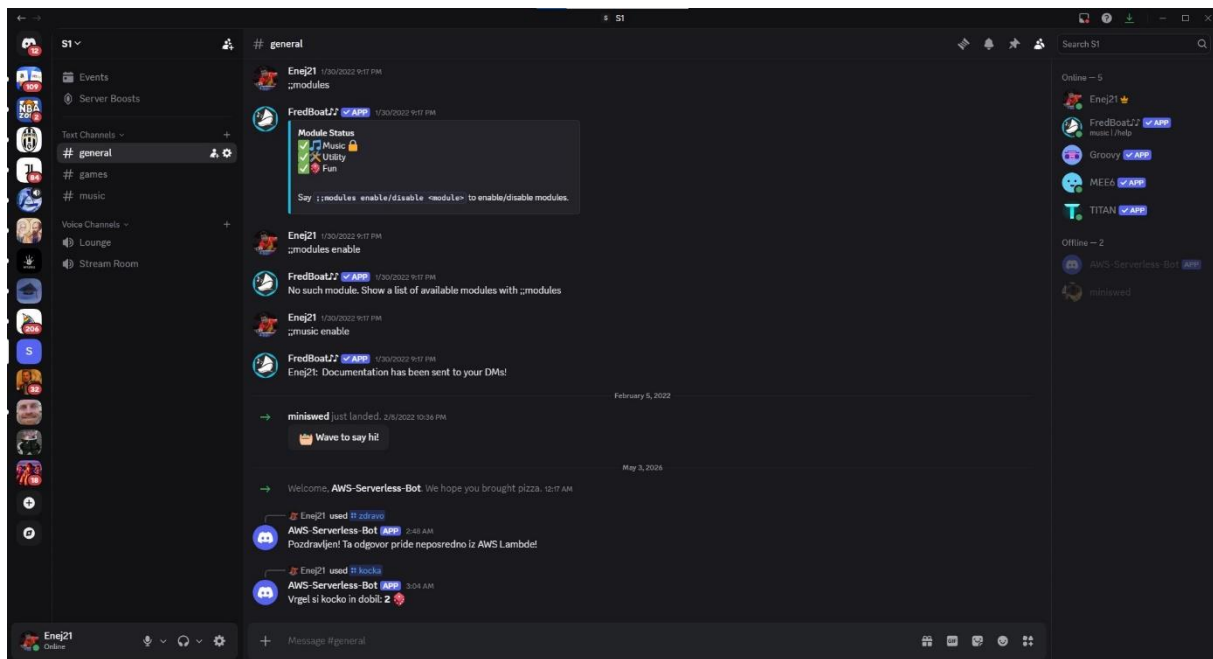


```
1 import requests
2
3 APP_ID = "1500249955884073202"
4 GUILD_ID = "93674220858087356"
5 BOT_TOKEN = "MTUwMDI0OTk1NTg4NDMzMzI2Mg.6NMDV.BWz2GAP2Za3JFpGArPqQ0net_CkZAYyGWhBI"
6
7 url = f"https://discord.com/api/v10/applications/{APP_ID}/guilds/{GUILD_ID}/commands"
8
9 json_data = [
10     {
11         "name": "zdravo",
12         "description": "Bot te pozdravi iz AWS-a",
13         "type": 1
14     },
15     {
16         "name": "kocka",
17         "description": "Vržeš kocko (1-6)",
18         "type": 1
19     }
20 ]
21
22 headers = {"Authorization": f"Bot {BOT_TOKEN}"}
23 response = requests.put(url, headers=headers, json=json_data)
24
25 print(response.status_code)
26 print(response.json())
```

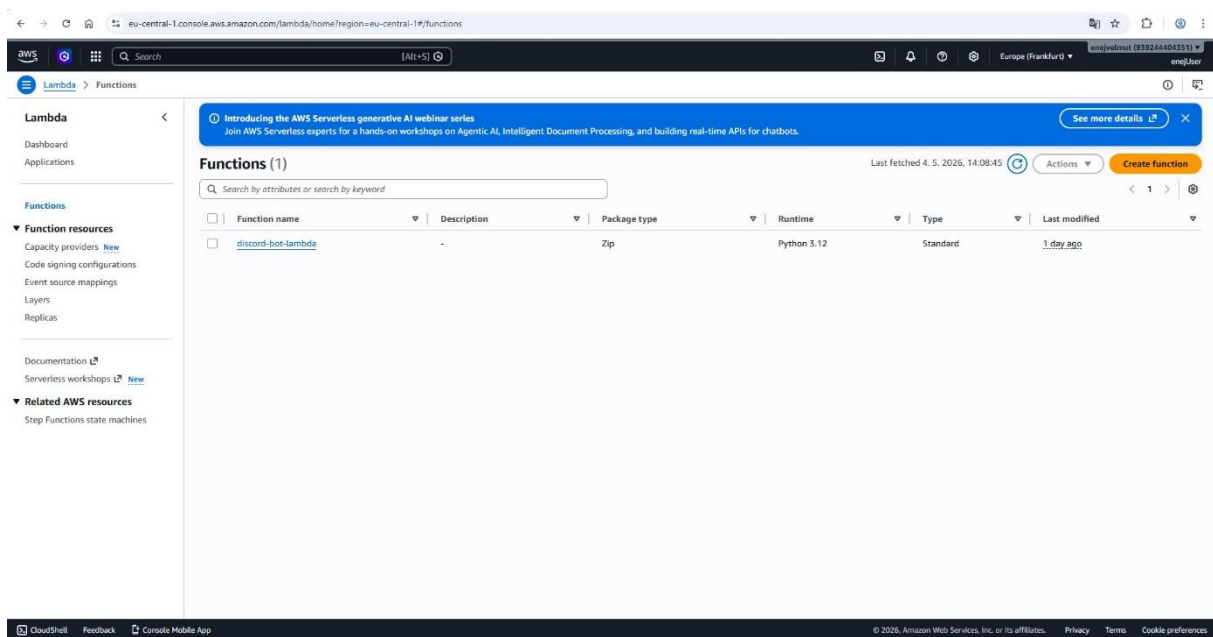
Run register.py

```
"I:\\Alma Mater\\RAČUNALNIŠTVO V OBLAKU IN MOBILE REŠITVE\\projektna\\.venv\\Scripts\\python.exe" "I:\\Alma Mater\\RAČUNALNIŠTVO V OBLAKU IN MOBILE REŠITVE\\projektna\\discord-aws-bot\\register.py"
200
[{"id": "1500297279293227129", "application_id": "1500249955884073202", "version": "1500297279293227130", "default_member_permissions": None, "type": 1, "name": "zdravo", "name_localizations": None, "description": "Bot te p"}
Process finished with exit code 0
```

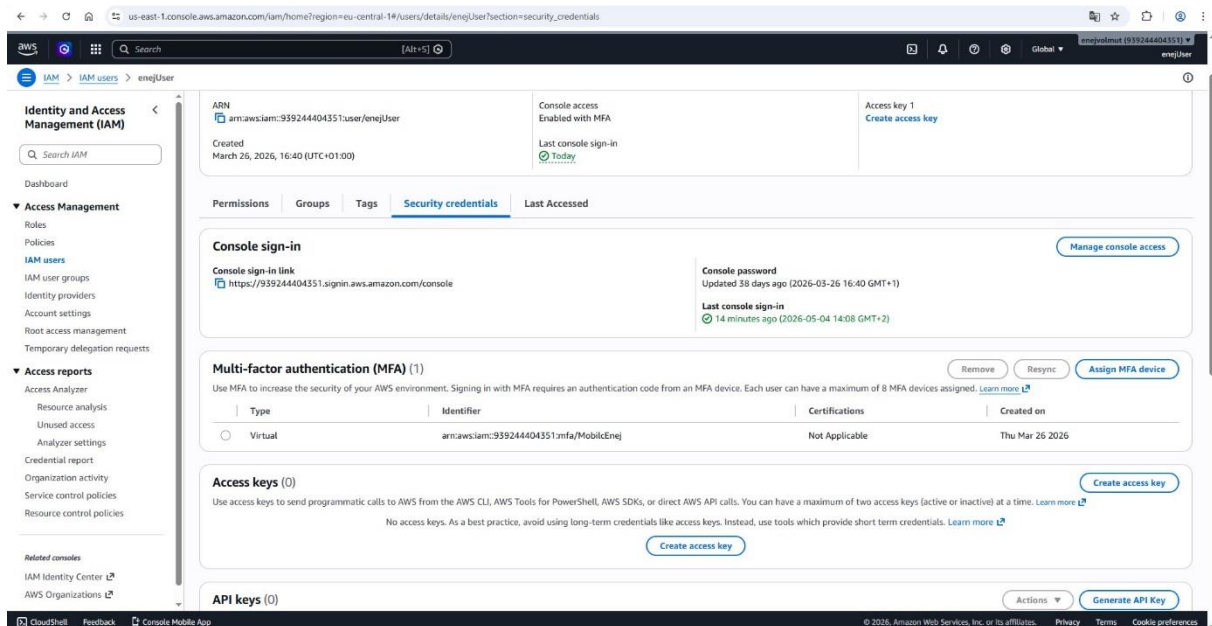




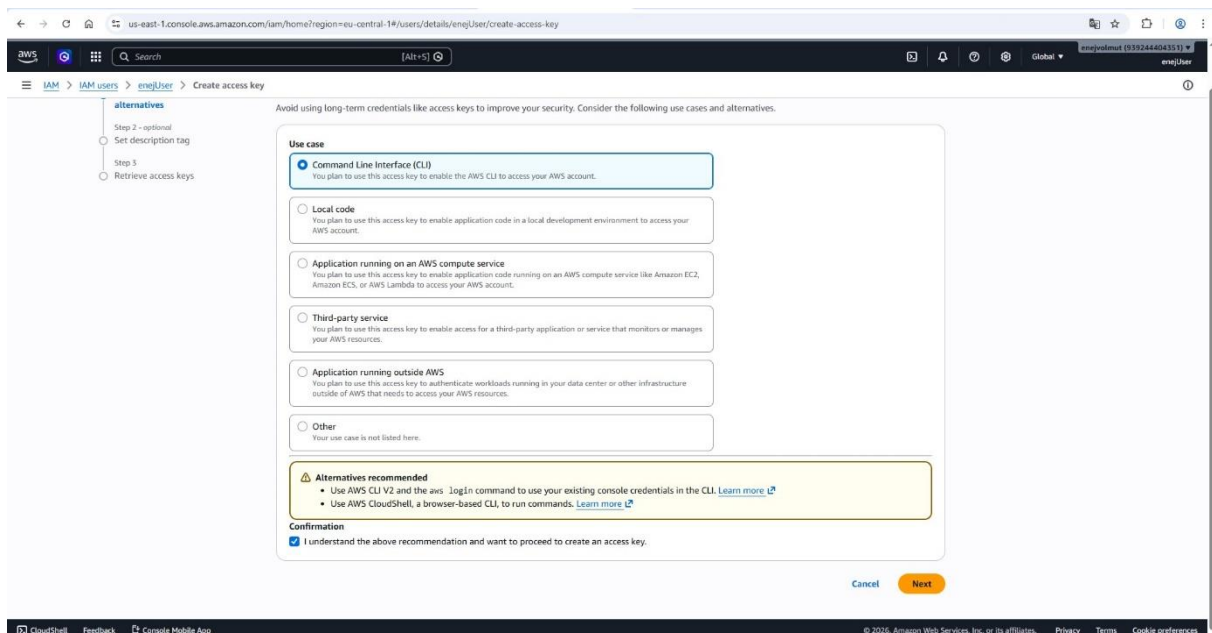
V nadzorni plošči AWS Lambda sem najprej preveril stanje svoje aktivne funkcije discord-bot-lambda, da sem potrdil pripravljenost za nadaljnje povezovanje.

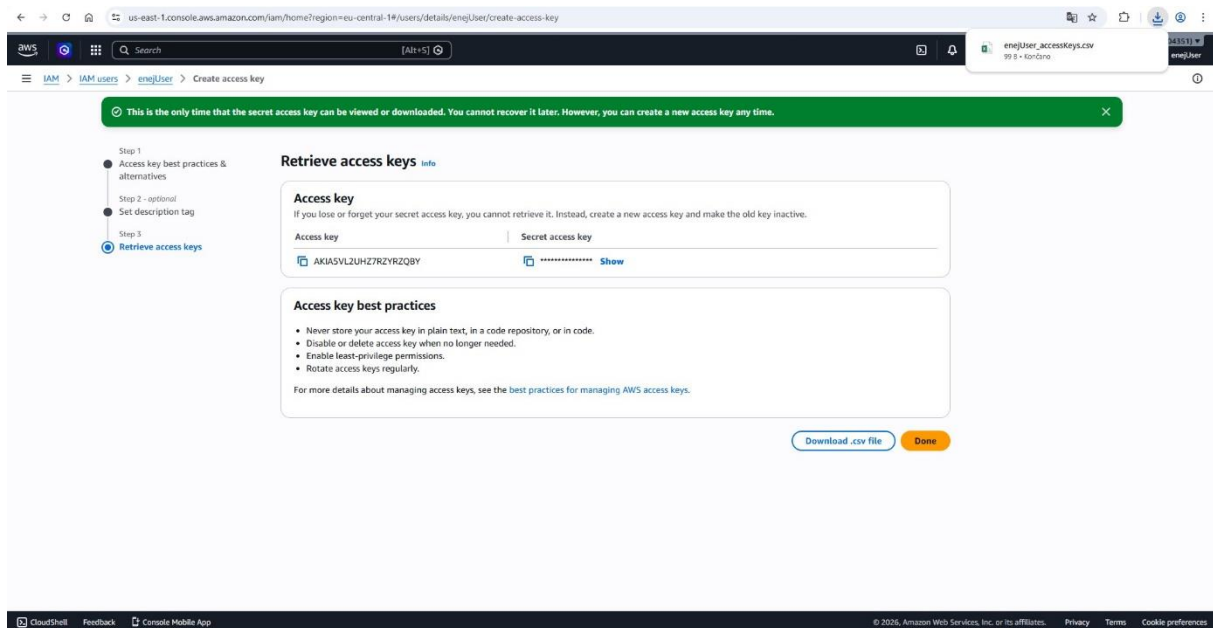


V razdelku IAM sem pod svojimi uporabniškimi nastavitvami poiskal zavihek z varnostnimi poverilnicami, kjer sem začel postopek ustvarjanja novih ključev.

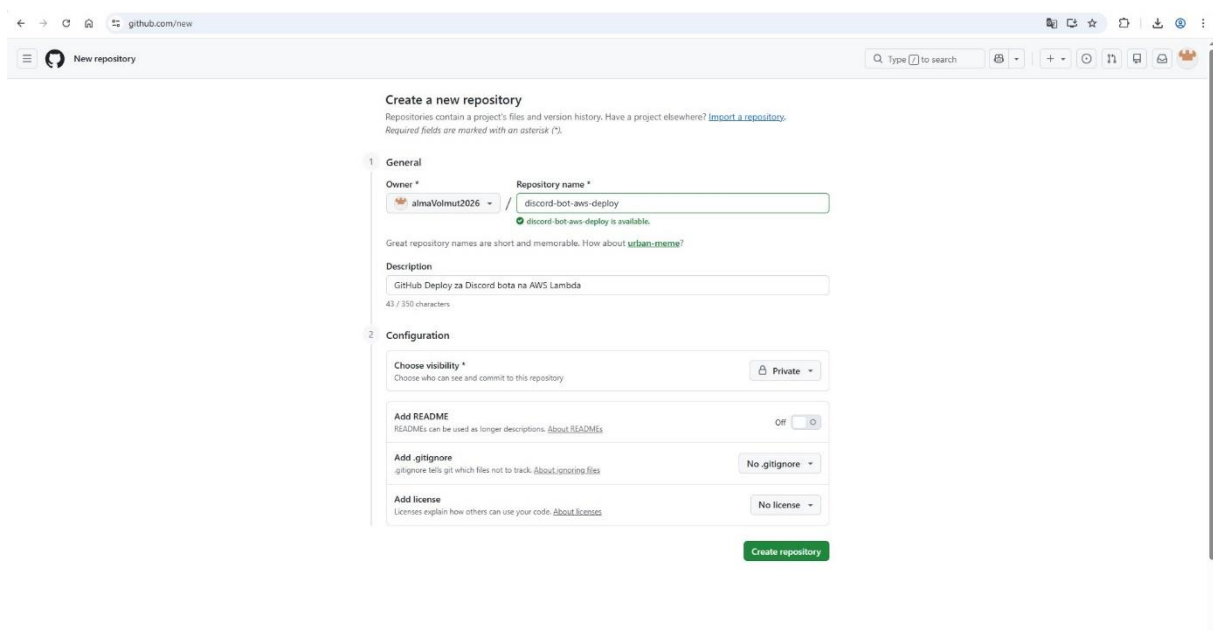


Pri izbiri namena uporabe ključa sem označil možnost Command Line Interface, saj sem potreboval poverilnice za zunanjo avtentikacijo, ki mi bodo kasneje v GitHubu omogočile samodejno posodabljanje kode neposredno iz repozitorija. Prenesel sem si .csv datoteko, kjer sta bila zapisana Access Key ID in Secret Access Key.

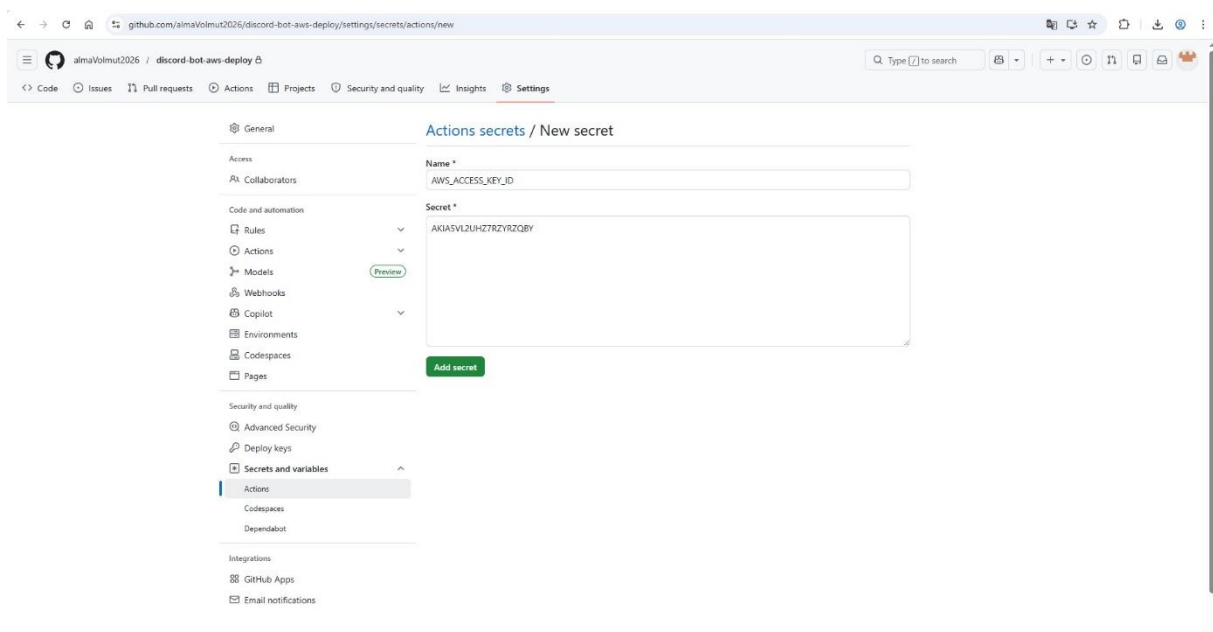
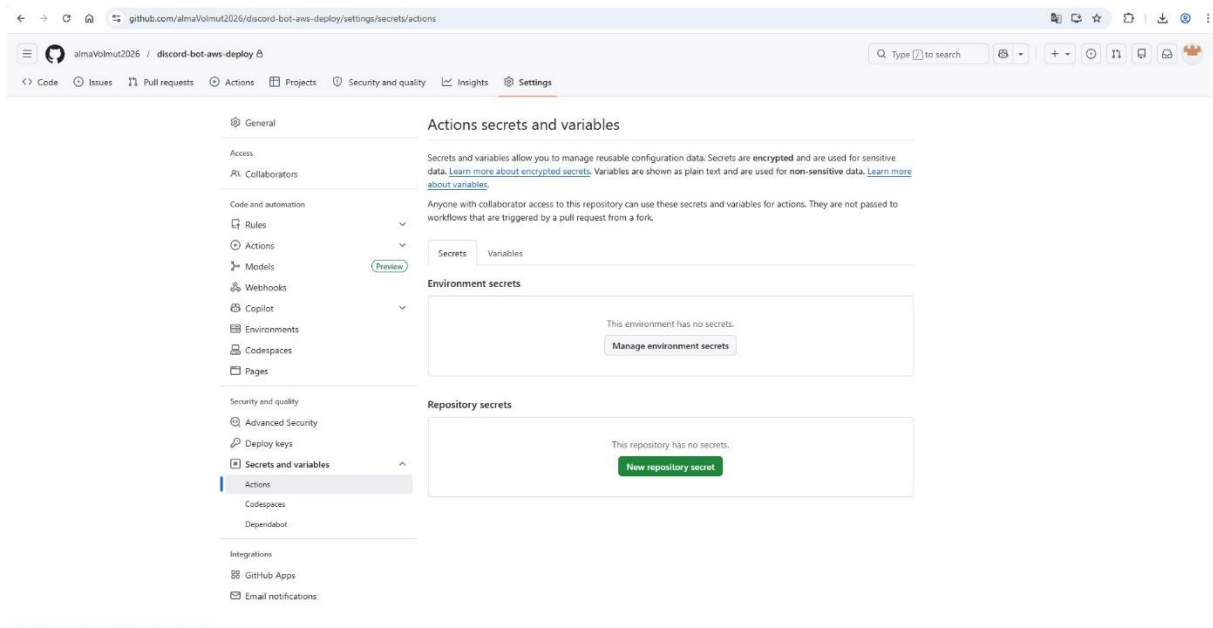


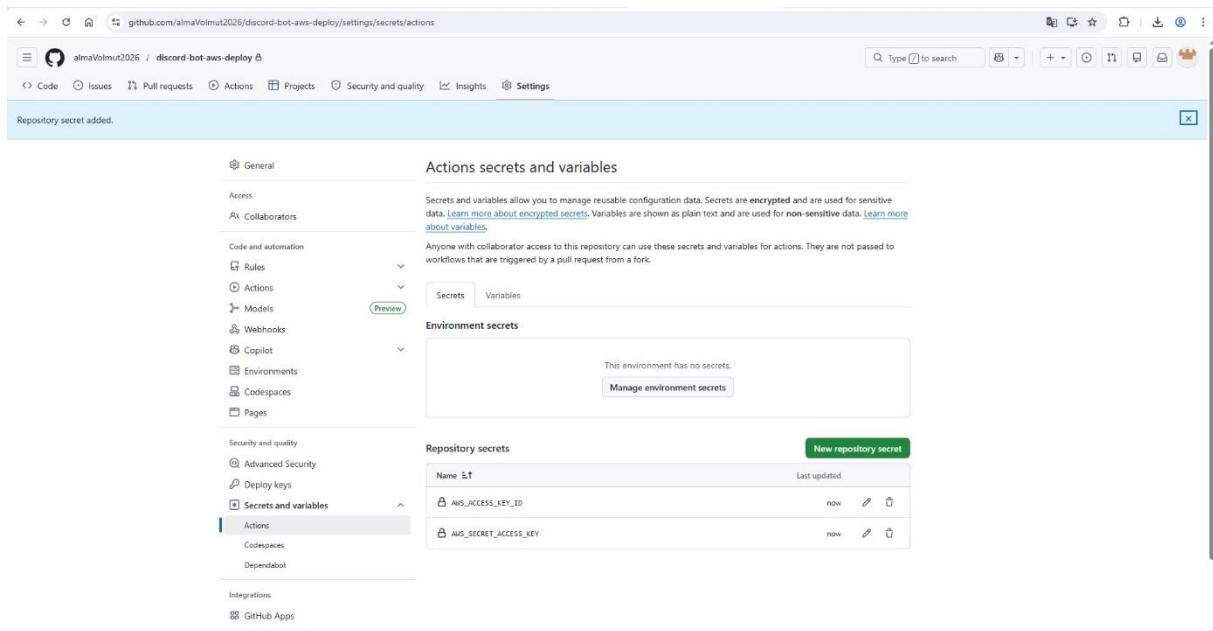
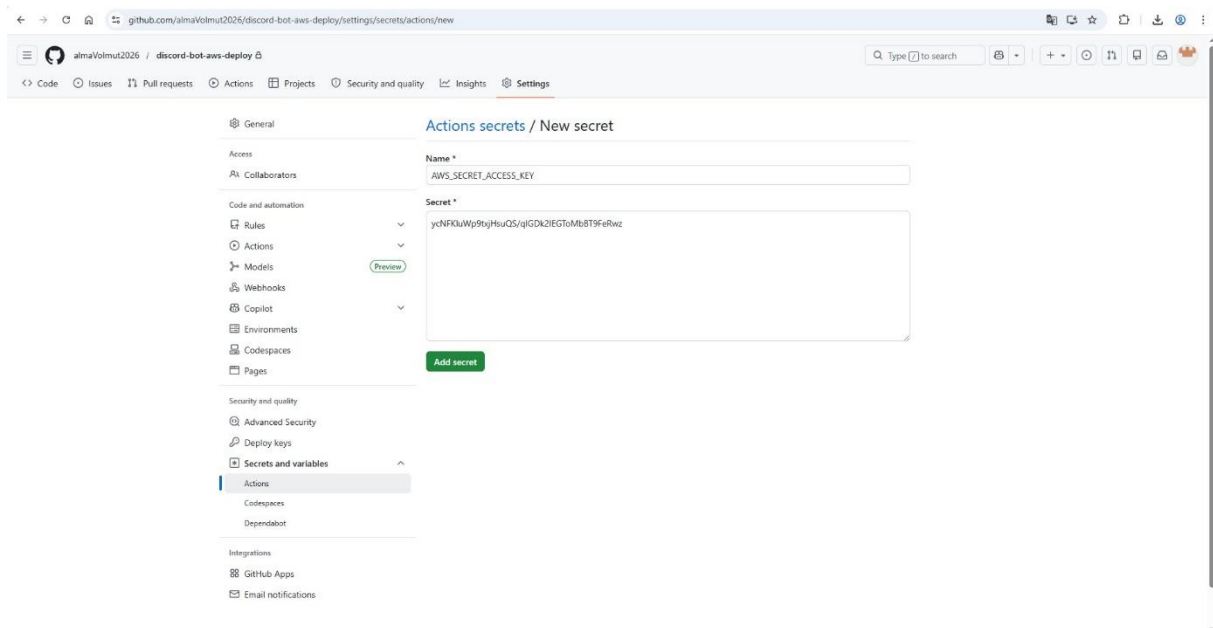


Na platformi GitHub sem najprej ustvaril nov zasebni repozitorij z imenom discord-bot-aws-deploy.

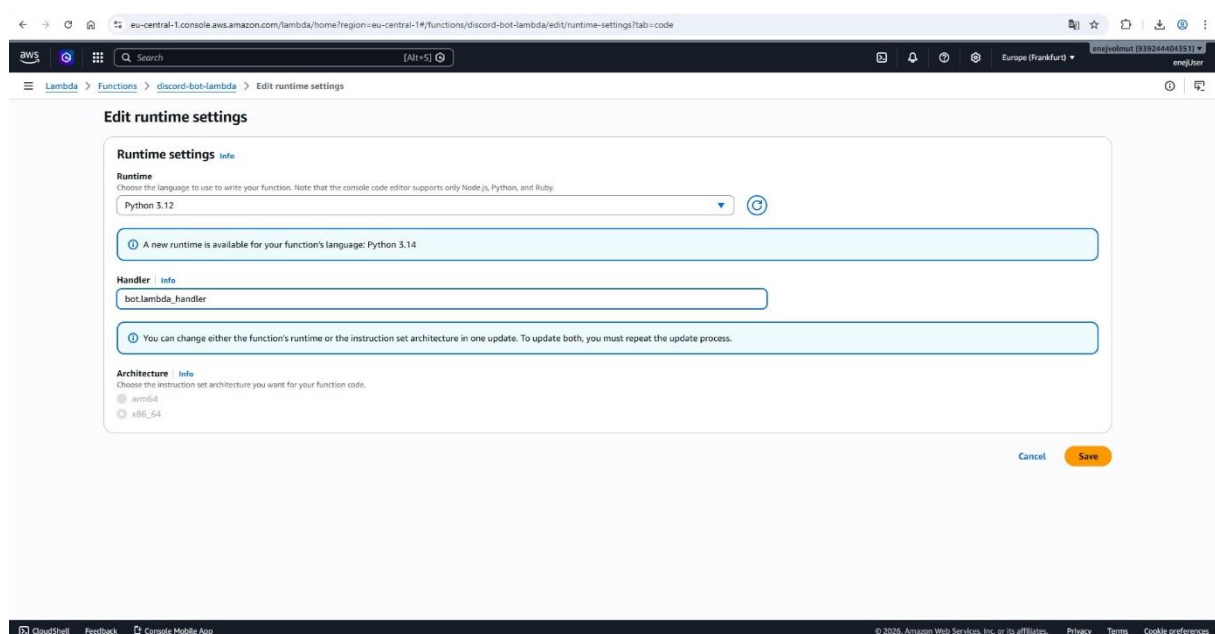
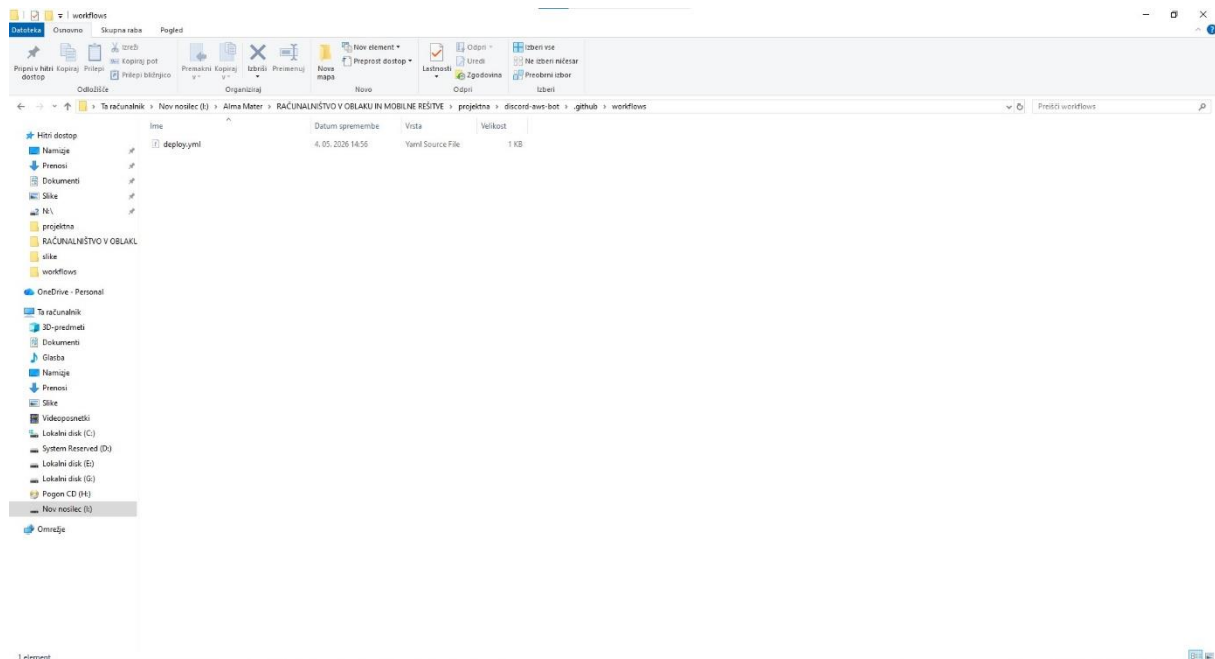


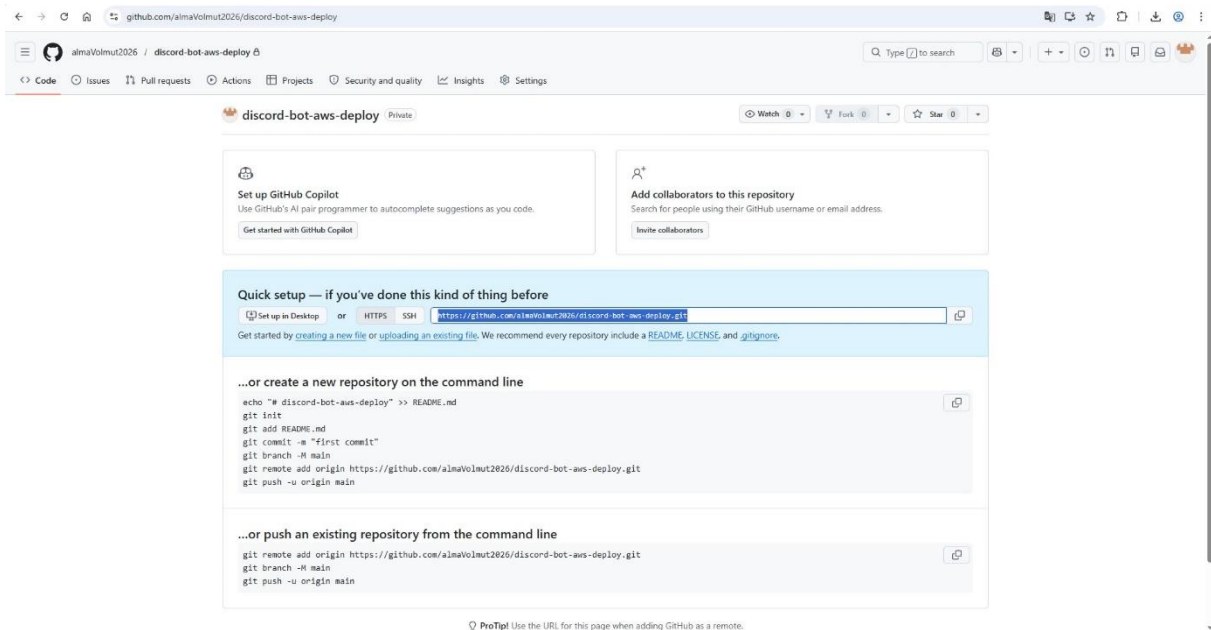
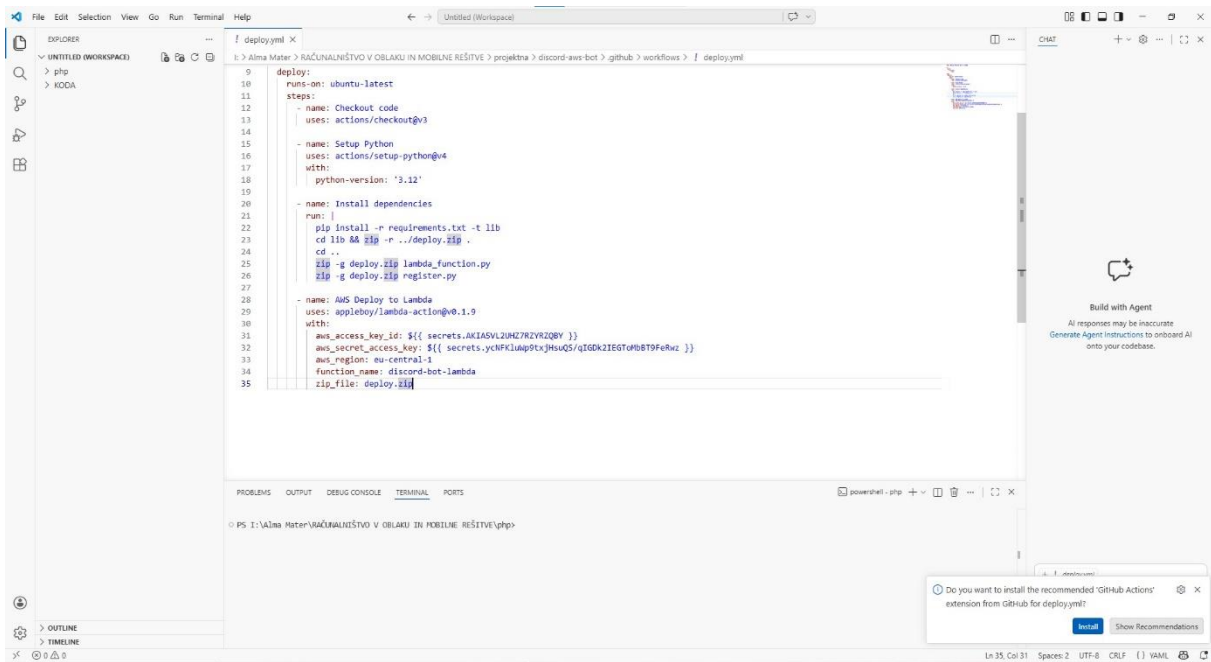
Za zagotovitev varne povezave sem v nastavitvah repozitorija pripravil prostor za vnos svojih AWS poverilnic, ki so nujne za delovanje avtomatiziranih akcij.





V korensko mapo projekta, kjer sem že imel datoteke bot.py, register.py in requirements.txt, sem vstavil konfiguracijsko datoteko deploy.yml z vsemi potrebnimi parametri za avtomatizacijo, hkrati pa sem v nastavitvah AWS Lambde ročno posodobil handler na bot.lambda_handler, da je sistem pravilno prepoznal vstopno točko moje kode in omogočil nemoteno izvajanje bota v oblaku.

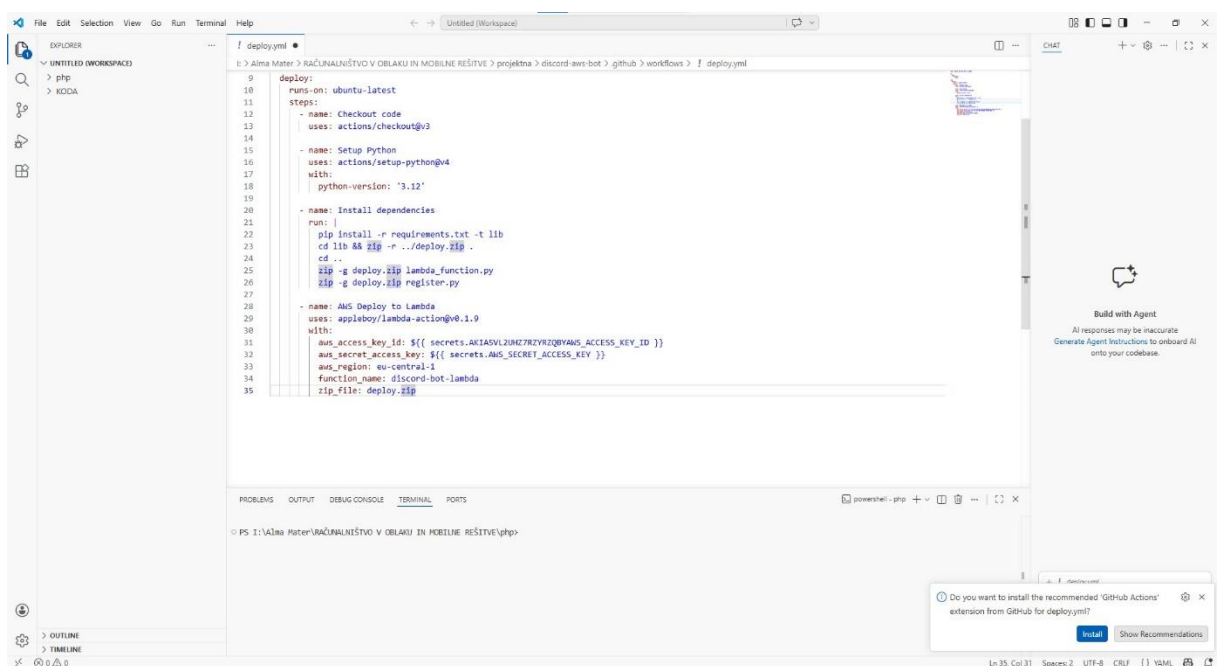
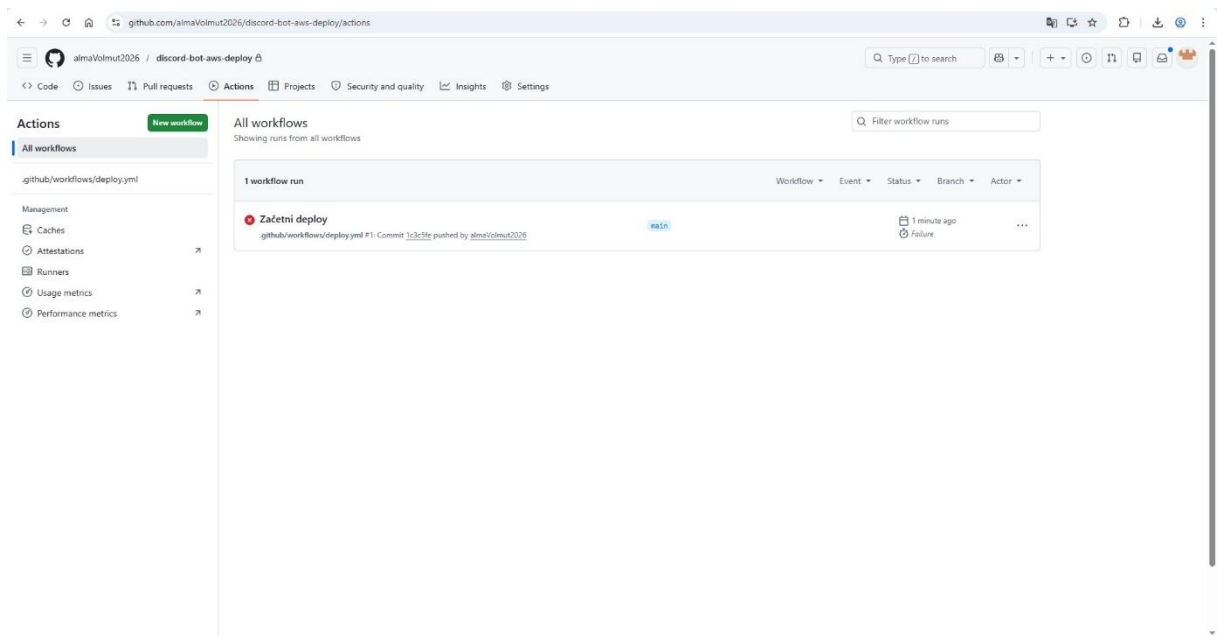




V terminalu Git Bash sem nadaljeval proces z inicializacijo lokalnega repozitorija in povezavo z oddaljenim naslovom na GitHubu, nato pa sem preko serije ukazov pripravil vse datoteke, vključno z bot.py in konfiguracijo za deploy, jih potrdil z začetnim commitom ter jih s končnim ukazom push uspešno poslal v oblak, s čimer sem sprožil avtomatiziran postopek namestitve.

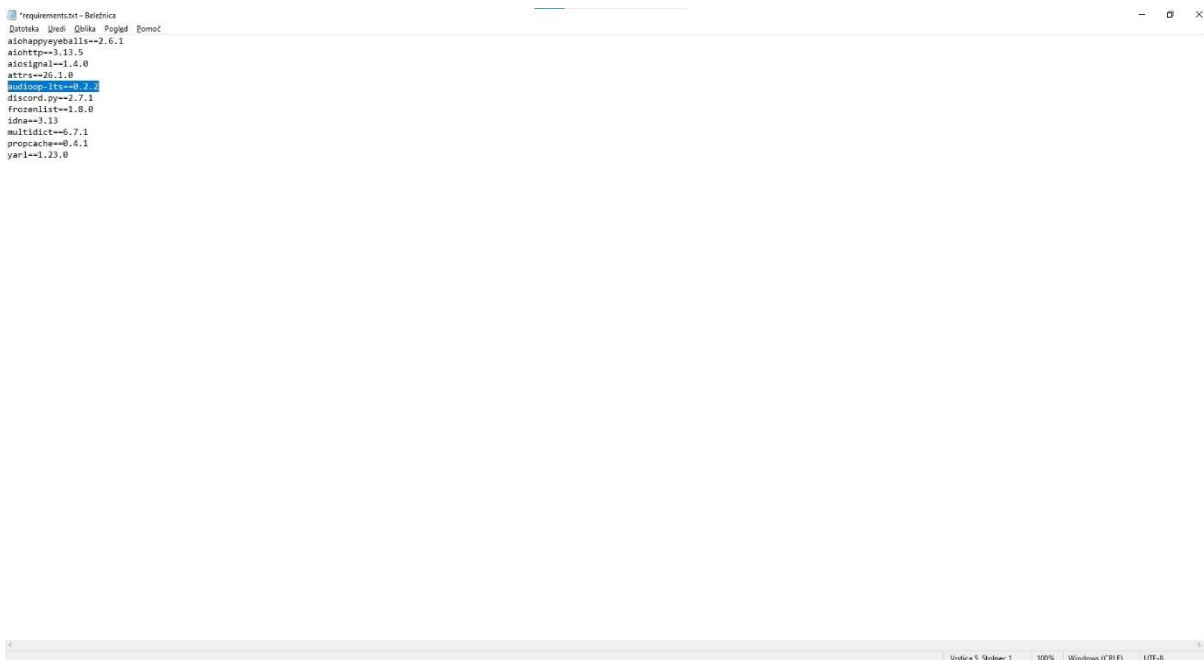
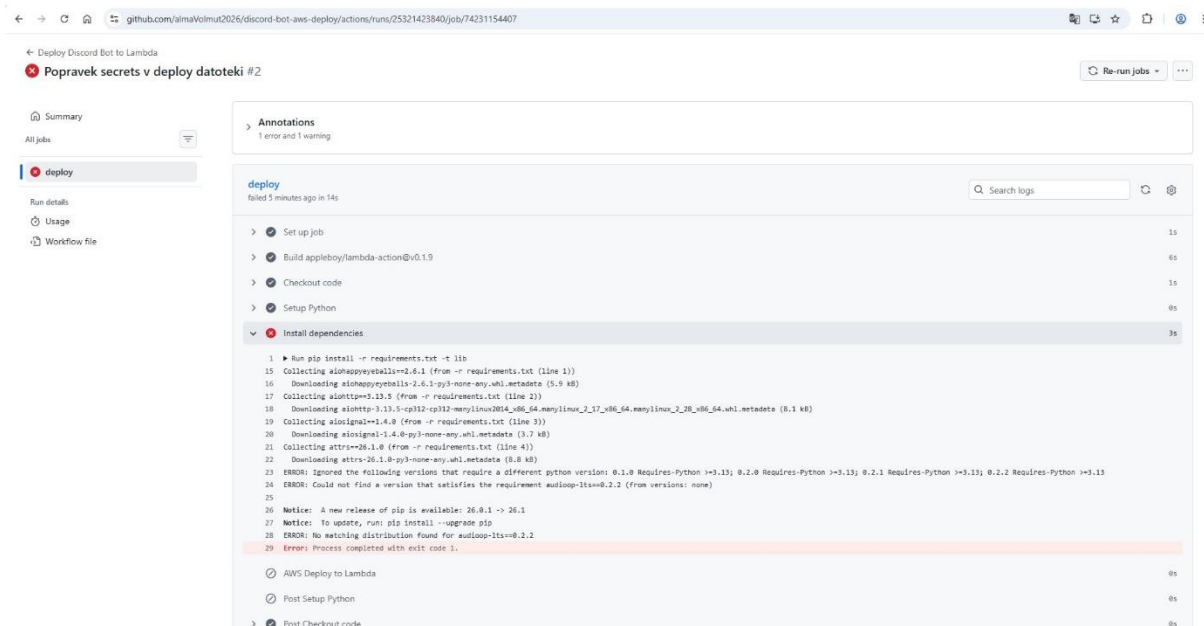
```
MINGW64 ~/Alma Mater/RACUNALNIŠTVO V OBLAKU IN MOBILE RESITVE/projekta/discord-aws-bot
Assuredesktop@HP45801: WINGIDEA /~/Alma Mater/RACUNALNIŠTVO V OBLAKU IN MOBILE RESITVE/projekta/discord-aws-bot
$ git init
Initialized empty Git repository in I:/Alma Mater/RACUNALNIŠTVO V OBLAKU IN MOBILE RESITVE/projekta/discord-aws-bot/.git/
Assuredesktop@HP45801: WINGIDEA /~/Alma Mater/RACUNALNIŠTVO V OBLAKU IN MOBILE RESITVE/projekta/discord-aws-bot (master)
$ git remote add origin https://github.com/AlmaMater2026/discord-bot-aws-deploy.git
Assuredesktop@HP45801: WINGIDEA /~/Alma Mater/RACUNALNIŠTVO V OBLAKU IN MOBILE RESITVE/projekta/discord-aws-bot (master)
$ git branch -M main
Assuredesktop@HP45801: WINGIDEA /~/Alma Mater/RACUNALNIŠTVO V OBLAKU IN MOBILE RESITVE/projekta/discord-aws-bot (main)
$ git add .
Assuredesktop@HP45801: WINGIDEA /~/Alma Mater/RACUNALNIŠTVO V OBLAKU IN MOBILE RESITVE/projekta/discord-aws-bot (main)
$ git commit -m "Začetni deploy"
[main (root-commit) 1c1c3f6] Začetni deploy
 1 files changed, 38 insertions(+), 0 deletions(-)
 create mode 100644 .github/workflows/deploy.yml
 create mode 100644 bot.py
 create mode 100644 requirements.txt
Assuredesktop@HP45801: WINGIDEA /~/Alma Mater/RACUNALNIŠTVO V OBLAKU IN MOBILE RESITVE/projekta/discord-aws-bot (main)
$ git push -u origin main
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 12 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (8/8), 3.72 KiB | 897.00 KiB/s, done.
Total 8 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com:AlmaMater2026/discord-bot-aws-deploy.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
Assuredesktop@HP45801: WINGIDEA /~/Alma Mater/RACUNALNIŠTVO V OBLAKU IN MOBILE RESITVE/projekta/discord-aws-bot (main)
$
```

Moj prvi poskus avtomatizirane objave je bil sprva neuspešen zaradi napake v datoteki `deploy.yml`, saj je GitHub Actions javil nepričakovane simbole pri definiciji skrivnih ključev, zato sem v urejevalniku VS Code pregledal kodo in popravil napačne zapise za dostopni ključ. Po popravku sem v terminalu Git Bash ponovil postopek pošiljanja sprememb, s čimer sem posodobil repozitorij.

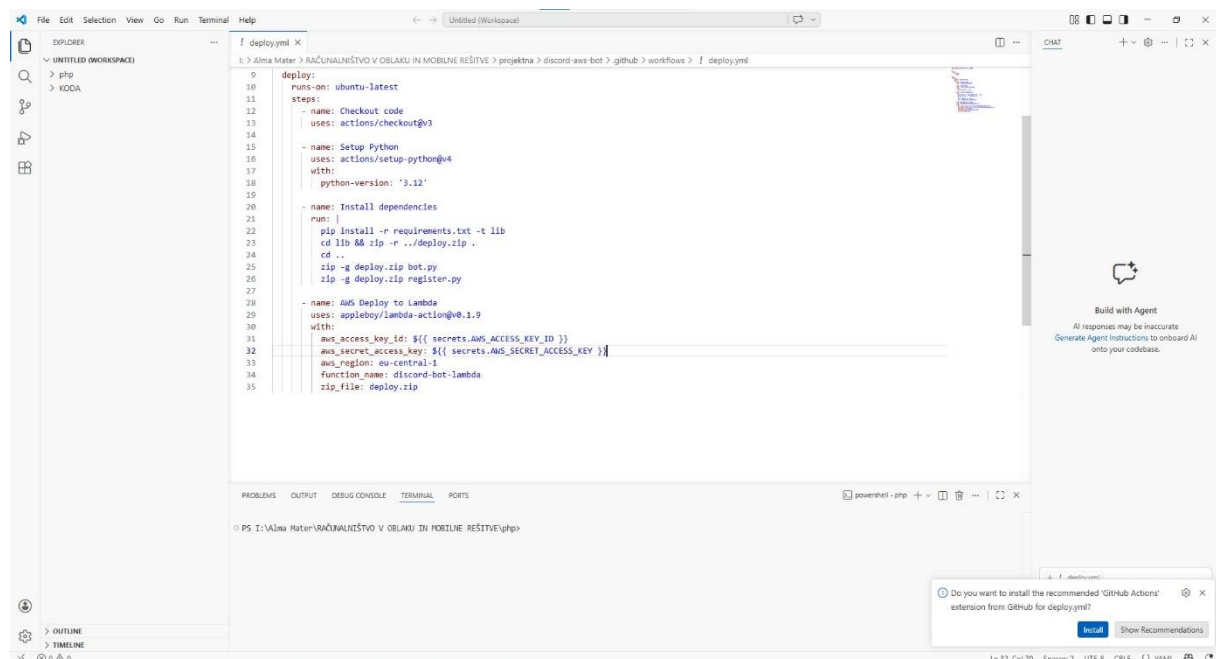
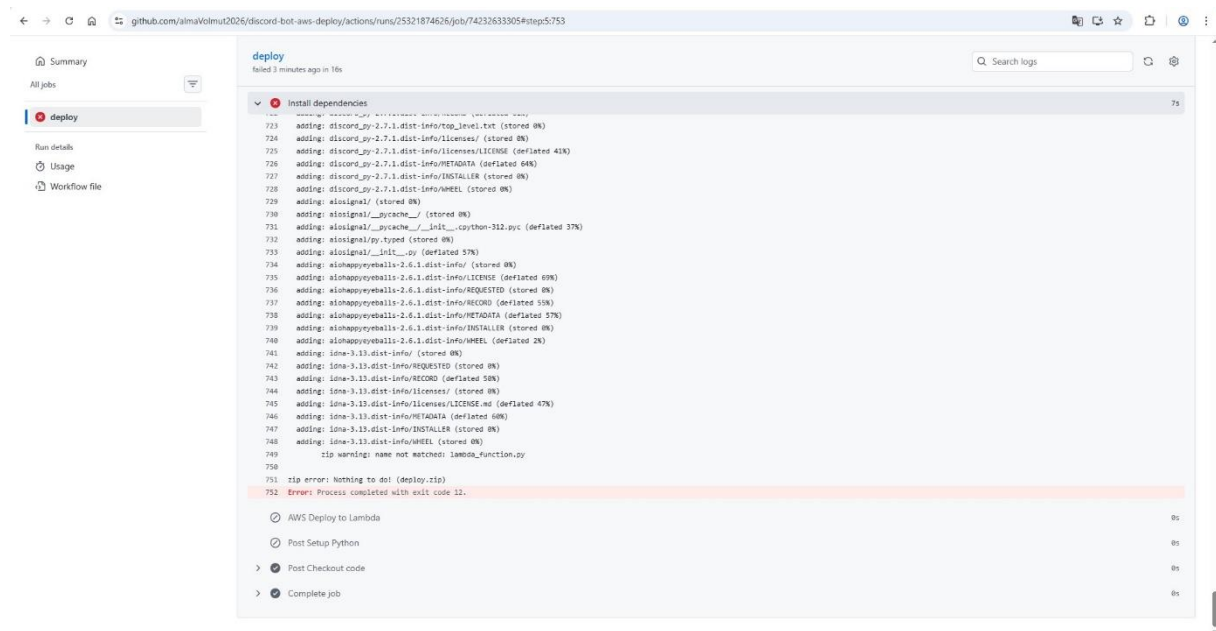


```
BRUGW04-uAlma Mater-RACUNALNOST V OBLAKU IN MOBILNE RESITVE/projektna/discord-aws-bot
KousRDESKTOP-HP4E9B3 WING064 /c/Alma Mater/RACUNALNOST V OBLAKU IN MOBILNE RES
ITVE/projektna/discord-aws-bot
$ git init
Initialized empty Git repository in E:/Alma Mater/RACUNALNOST V OBLAKU IN MOBI
LE RESITVE/projektna/discord-aws-bot/.git/
KousRDESKTOP-HP4E9B3 WING064 /c/Alma Mater/RACUNALNOST V OBLAKU IN MOBILNE RES
ITVE/projektna/discord-aws-bot (master)
$ git remote add origin https://github.com/AlmaNout2016/discord-bot-aws-deploy.git
KousRDESKTOP-HP4E9B3 WING064 /c/Alma Mater/RACUNALNOST V OBLAKU IN MOBILNE RESITVE/pr
ojektna/discord-aws-bot (master)
$ git branch -M main
KousRDESKTOP-HP4E9B3 WING064 /c/Alma Mater/RACUNALNOST V OBLAKU IN MOBILNE RESITVE/pr
ojektna/discord-aws-bot (main)
$ git add .
KousRDESKTOP-HP4E9B3 WING064 /c/Alma Mater/RACUNALNOST V OBLAKU IN MOBILNE RESITVE/pr
ojektna/discord-aws-bot (main)
$ git commit -m "začetni deploy"
[main f0c0c5b] 1dc3cf9: Začetni deploy
4 files changed, 36 insertions(+)
create mode 100644 .github/workflows/deploy.yml
create mode 100644 bot.py
create mode 100644 register.py
create mode 100644 requirements.txt
KousRDESKTOP-HP4E9B3 WING064 /c/Alma Mater/RACUNALNOST V OBLAKU IN MOBILNE RESITVE/pr
ojektna/discord-aws-bot (main)
$ git push -u origin main
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 12 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (8/8), 1.75 KiB | 897.00 KiB/s, done.
Total 8 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
to https://github.com:AlmaNout2016/discord-bot-aws-deploy.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
KousRDESKTOP-HP4E9B3 WING064 /c/Alma Mater/RACUNALNOST V OBLAKU IN MOBILNE RESITVE/pr
ojektna/discord-aws-bot (main)
$ git add .
KousRDESKTOP-HP4E9B3 WING064 /c/Alma Mater/RACUNALNOST V OBLAKU IN MOBILNE RESITVE/projektna/discord-aws-bot (main)
$ git commit -m "Popravljam sekreta v deploy datoteki"
[main f1ee22c] Popravljam sekreta v deploy datoteki
1 file changed, 2 insertions(+), 2 deletions(-)
KousRDESKTOP-HP4E9B3 WING064 /c/Alma Mater/RACUNALNOST V OBLAKU IN MOBILNE RESITVE/projektna/discord-aws-bot (main)
$ git push origin main
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 12 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (8/8), 330 bytes | 330.00 KiB/s, done.
Total 8 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
to https://github.com:AlmaNout2016/discord-bot-aws-deploy.git
1dc3cf9..f1ee22c main -> main
KousRDESKTOP-HP4E9B3 WING064 /c/Alma Mater/RACUNALNOST V OBLAKU IN MOBILNE RESITVE/projektna/discord-aws-bot (main)
$
```


Kljub popravku datoteke proces ponovno ni uspel, saj se je v fazi nameščanja pojavila nova napaka, ker sistem ni našel ustrezne različice knjižnice audioop-lts za uporabljeno verzijo Pythona. Da bi odpravil to težavo, sem odprl datoteko requirements.txt in poiskal problematično vrstico, ki je povzročala prekinitev avtomatiziranega postopka v oblaku.



Po odstranitvi knjižnice je sistem našel še zadnjo napako pri pakiranju arhiva, saj sistem ni našel datoteke `lambda_function.py`, ker sem v kodi še vedno imel napisano staro ime. Odprl sem `deploy.yml` in popravil ukaze za zipanje, nato pa sem v terminalu Git Bash popravke poslal v repozitorij. Tokrat je avtomatizacija v GitHub Actions stekla brez težav in po 27 sekundah sem prejel zeleno potrditev o uspešni namestitvi (Success), s čimer je bil moj Discord bot objavljen na AWS Lambda.



The screenshot shows the GitHub Actions interface for the repository 'almaVolmu2026 / discord-bot-aws-deploy'. The workflow 'Deploy Discord Bot to Lambda' is selected, showing a list of 5 workflow runs. The runs are as follows:

Run Name	Commit	Status	Branch	Actor	Time
Popravek secrets v deploy kodi	Deploy Discord Bot to Lambda #5: Commit e5325e pushed by almaVolmu2026	Success	main	almaVolmu2026	14 minutes ago
Popravek v deploy kodi	Deploy Discord Bot to Lambda #6: Commit 38fa36 pushed by almaVolmu2026	Success	main	almaVolmu2026	14 minutes ago
Popravek requirements datoteke	Deploy Discord Bot to Lambda #3: Commit 1cd32a pushed by almaVolmu2026	Success	main	almaVolmu2026	10 minutes ago
Popravek secrets v deploy datoteki	Deploy Discord Bot to Lambda #2: Commit f1ee22 pushed by almaVolmu2026	Success	main	almaVolmu2026	19 minutes ago
Začetni deploy	Deploy Discord Bot to Lambda #1: Commit 1c3c56 pushed by almaVolmu2026	Failure	main	almaVolmu2026	47 minutes ago

Za konec sem preveril še stanje v konzoli AWS Lambda, kjer je bilo pri moji funkciji izpisano, da je bila koda uspešno posodobljena pred 4 minutami, kar je potrdilo, da avtomatizacija deluje.

The screenshot shows the AWS Lambda console for the 'discord-bot-lambda' function. The function is listed with the following details:

Function name	Description	Package type	Runtime	Type	Last modified
discord-bot-lambda	-	Zip	Python 3.12	Standard	4 minutes ago

4. ZAKLJUČEK

Z izvedbo te projektne naloge sem se naučil, kako povezati različne platforme, kot so GitHub, AWS in Discord, v enoten, avtomatiziran sistem. Skozi praktično delo sem spoznal prednosti brezstrežnega računalništva z uporabo storitve AWS Lambda, ki omogoča izvajanje kode brez potrebe po upravljanju lastne infrastrukture. Kljub začetnim težavam pri konfiguraciji datoteke `deploy.yml` in usklajevanju knjižnic, mi je uspelo doseči, da bot deluje samostojno v oblaku.

5. VIRI

Amazon Web Services. (2026). Pridobljeno iz AWS Identity and Access Management (IAM) user guide: <https://docs.aws.amazon.com/IAM/latest/UserGuide/>

Amazon Web Services. (2026). Pridobljeno iz AWS Lambda developer guide: <https://docs.aws.amazon.com/lambda/>

Discord. (2026). Pridobljeno iz Discord developer portal documentation: <https://discord.com/developers/docs/intro>

GitHub. (2026). Pridobljeno iz GitHub Actions documentation: <https://docs.github.com/en/actions>

GitHub. (2026). Pridobljeno iz Git Bash - Git for Windows: <https://gitforwindows.org/>

JetBrains. (2026). Pridobljeno iz PyCharm help & documentation: <https://www.jetbrains.com/pycharm/documentation/>

Microsoft. (2026). Pridobljeno iz Visual Studio Code documentation: <https://code.visualstudio.com/docs>

Python Software Foundation. (2026). Pridobljeno iz Python 3.12.3 documentation: <https://docs.python.org/3/>

Rapptz. (2026). Pridobljeno iz discord.py documentation (Version 2.3): <https://discordpy.readthedocs.io/>

YAML.org. (2021). Pridobljeno iz YAML Ain't Markup Language (YAML) version 1.2: <https://yaml.org/spec/1.2.2/>